المدرسة الحسنية للأشغال العمومية
ECOLE HASSANIA DES TRAVAUX PUBLICS

وزارة التجهيز والماء
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵜⵣⵔⴼⵜ ⴷ ⵓⵎⴰⵢ
MINISTERE DE L'EQUIPEMENT
ET DE L'EAU

Professional Initiation Internship Report

Internship duration: 2 months

Calibration of AI Weather Models Using Machine Learning

Carried out by the first-year engineering student in Mathematics and Systems Engineering:

IDRISSI MELIANI Hamza

Under the supervision of:

Dr. ALAOUI Badreddine

Senior Data Scientist

- Host organisation:

المديرية العامة للأرصاد الجوية
DIRECTION GENERALE DE LA METEOROLOGIE
MAROC METEO

Academic year: **2025/2026**

Acknowledgements

I would like to thank my supervisor, **Dr. ALAOUI Badreddine**, for his guidance during these 2 months. Because the internship was carried out remotely, the subject details, the data, and the answers to my questions all came through him. He replied patiently and gave me the autonomy to test my own ideas and explore different solutions. That professional freedom was an essential part of my learning experience.

I also thank the General Directorate of Meteorology (DGM) for proposing the subject and for giving me access to the dataset on which the study is based. Without this dataset, the study would not have been possible.

Finally, I thank the administration and the teachers of the **Hassania School of Public Works** for making this first professional experience possible.

Abstract

This report presents the work I carried out during a two-month initiation internship at the General Directorate of Meteorology (DGM). The subject was the calibration of AI weather models using machine learning. In meteorology, calibration means correcting the systematic errors of a forecast after the model has produced it. My task was to measure these errors for eight AI weather models over Morocco, and then reduce them using machine learning.

The dataset contains 261,082 forecast and observation pairs, collected at 34 Moroccan weather stations between September 2025 and March 2026, for five weather variables and forecast horizons from 0 to 72 hours. An exploratory analysis of the bias showed that the hour of the day explains a large part of the error for three models (AIFS, DLWP and GraphCast), while station location matters more for the five others. Four algorithms were then trained to predict the bias of the 2 m temperature forecast: a multi-layer perceptron, a support vector regression, a random forest and LightGBM.

The corrected forecasts were compared with the original ones on the most recent 20% of the data. The error decreased for all eight models. AIFS improved the most, with a reduction of 44.7% in RMSE for the single-variable configuration and 49.0% when other predicted variables were added. The models that were already accurate improved by about 15% to 20%. Precipitation remained difficult to correct because more than 98% of the observations contain no rain.

Keywords: weather forecast calibration, bias correction, AI weather models, machine learning, statistical post-processing.

Contents

Acknowledgements 1

Abstract 2

List of Abbreviations 8

Introduction 10

1 The host organisation 12

1.1 History and missions . . . 12
1.2 Internal organisation . . . 13
1.3 Research and innovation . . . 14
1.4 Project Context . . . 14

2 Literature review 16

2.1 From physics models to AI weather models . . . 16
2.2 Why a good forecast still needs a correction . . . 17
2.3 Grid resolution and the representativeness error . . . 17
2.4 Statistical post-processing before machine learning . . . 18
2.5 Neural networks in meteorological post-processing . . . 19
2.6 Overconfidence and probability calibration in deep learning . . . 19
2.7 Algorithm choice and the size of the correction dataset . . . 20
2.8 Summary of the background . . . 20

3 Datasets and methodology 22

3.1 Description of the data . . . 22
3.2 The five weather variables . . . 25
3.3 The station network . . . 26
3.4 Data preparation . . . 26
3.5 Feature engineering . . . 27

3.6 Forecast times: run time, lead time and valid time . . . 28
3.7 Theoretical background of the four algorithms . . . 29
3.8 Formulas used in the analysis . . . 30
3.9 How the bias was analysed . . . 31
3.10 The calibration method . . . 32
3.11 Hyperparameter tuning . . . 33
3.12 A separate method for precipitation . . . 34
3.13 Evaluation metrics . . . 35

4 Results 37
4.1 Baseline accuracy of the raw AI forecasts . . . 37
4.2 The bias changes with the hour of the day . . . 39
4.3 The bias against lead time, temperature and location . . . 41
4.4 Features that explain the bias . . . 45
4.5 Calibration results for temperature . . . 45
4.6 Precipitation . . . 47
4.7 Consistency between the learned models and the exploratory analysis . . . 48
4.8 Summary of the results . . . 50
4.9 Comparison with published results . . . 50
4.10 Limitations . . . 51

Conclusion 52

Perspectives 53

Bibliography 55

A Supplementary tables 57
A.1 Full calibration results for temperature . . . 57
A.2 Precipitation results . . . 59
A.3 Selected code . . . 60
A.4 Selected hyperparameters . . . 63

List of Figures

1.1 Organisation of the General Directorate of Meteorology. The chart was redrawn from the internal organisation document of the host organisation [18]. 14

4.1 Distribution of temperature bias for each AI model. The vertical line marks zero bias. Histograms with different spreads can have the same mean bias but different RMSE values. 38

4.2 Models ranked by RMSE for each weather variable. Lower is better. 39

4.3 Mean temperature bias against the hour of the day, for the eight models. . 40

4.4 Distribution of the temperature bias by moment of the day, one panel per model. 40

4.5 Mean bias against the forecast horizon, for the five weather variables. . . . 42

4.6 Two views of the same problem: the models compress the range of temperatures, so cold days are predicted too warm and hot days too cold. 42

4.7 Temperature RMSE per station, all eight AI models and all lead times together, sorted from the easiest station to the hardest one. 43

4.8 Mean temperature bias per month, for the eight models. The record covers September 2025 to March 2026 and contains no summer month. 44

4.9 Absolute temperature bias against the distance from the station to the grid point, coloured by AI model. The two groups of points come from models with different grid geometries, which creates a misleading overall correlation. 44

4.10 Mutual information between each candidate feature and the temperature bias, one panel per AI model. 45

4.11 Percentage of rows with a precipitation bias above 0.1 mm, for the three models that forecast tp06. 47

4.12 Random Forest feature importance in Config A, one panel per AI model. . 49

4.13 LightGBM feature importance in Config A, one panel per AI model, on the same experiments as Figure 4.12. The two methods disagree about the binary time features. 49

P.1 Proposed operational chain for the correction. The upper row runs with every model run; the lower row runs on a slower cycle and keeps the correction up to date. 54

List of Tables

2.1 The eight AI weather models studied in this report. The architecture information comes from external sources [14, 13, 12, 15, 16, 17, 11]; the variables come from the dataset I received. 17

3.1 Main characteristics of the dataset used during the internship. 23

3.2 First rows of the merged file, for the DLWP model at lead time 0. The columns are shown in two blocks because they do not fit on one line. Values are taken from the file as received. 24

3.3 Number of rows available for each AI model and each weather variable. An empty cell means the variable is not available for that model in this dataset. 24

3.4 The five weather variables in the dataset. The last column gives the main reason a national weather service produces the variable, which is also the reason a bias in it has a cost. 25

3.5 The 34 stations of the dataset, grouped by the kind of place they sit in. The grouping is used in Section 4.3 to read the per-station errors. 26

3.6 Input features of the calibration models. Config A uses the first block only. Config B adds the other variables predicted by the same AI model, so its exact list depends on the model (Table 3.3). 28

3.7 Change in RMSE when the hyperparameters tuned on AIFS are applied to three other models. A positive value means the tuned version is better than the default one. 34

3.8 Contingency table for a yes or no forecast. The event here is “the precipitation bias of this row is larger than 0.1 mm in absolute value”. 36

4.1 Error of the raw temperature (t2m) forecasts, all stations and all lead times together. Sorted by RMSE. 38

4.2 Config A (single variable), temperature. Best algorithm per AI model, on the most recent 20% of the data. The raw RMSE is computed on these test rows only, which is why it differs from Table 4.1, computed over the whole period. 46

4.3 Config B (multi-parameter), temperature. Best algorithm per AI model. DLWP is missing because it provides temperature only. The raw RMSE is again computed on the test rows, here restricted to the runs where every variable of the model is available, so it differs from both other tables. . . . 47

A.1 Config A (single variable), temperature: all four algorithms for the eight AI models. 57

A.2 Config B (multi-parameter), temperature: all four algorithms for the seven AI models that publish more than one variable. 58

A.3 Precipitation (tp06) results. Precision, recall and F1 come from the classification step; MAE, RMSE and R^2 from the regression step. 60

A.4 Hyperparameters used for the calibration models. 63

A.5 Hyperparameters of the classifiers used in step 1 of the precipitation method. 64

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
AIFS	Artificial Intelligence Forecasting System (ECMWF)
ANOVA	Analysis of Variance
CRPS	Continuous Ranked Probability Score
DGM	General Directorate of Meteorology (Maroc Météo)
ECE	Expected Calibration Error
ECMWF	European Centre for Medium-Range Weather Forecasts
EHTP	Hassania School of Public Works
EMOS	Ensemble Model Output Statistics
FAR	False Alarm Ratio
FN	False Negative (a missed event)
FP	False Positive (a false alarm)
LGBM	LightGBM, a gradient boosting library
MAE	Mean Absolute Error
ML	Machine Learning
MLP	Multi-Layer Perceptron
msl	Mean sea level pressure
NWP	Numerical Weather Prediction
POD	Probability of Detection
RBF	Radial Basis Function, the kernel used by SVR and SVC
RF	Random Forest
RMSE	Root Mean Square Error
R^2	Coefficient of determination
SEGMA	*Service d'État Géré de Manière Autonome*
sp	Surface pressure
SVC	Support Vector Classifier

Abbreviation	Meaning
SVR	Support Vector Regression
SYNOP	Surface synoptic observation report
t2m	Air temperature at 2 metres
td2m	Dew point temperature at 2 metres
TN	True Negative
TP	True Positive
tp06	Total precipitation accumulated over 6 hours
UTC	Coordinated Universal Time
WMO	World Meteorological Organization

Introduction

Weather forecasting has undergone rapid transformation in recent years. Historically, forecasts relied exclusively on physics based models, which divide the atmosphere into a grid and solve fluid dynamics equations on supercomputing infrastructure. Since 2022, a new family of models has appeared. They are trained on decades of past weather data, learn patterns directly from that data, and produce a forecast in a few minutes on a single machine instead of hours on a supercomputer. These are AI weather models. ECMWF put its own model, AIFS, into operation on 25 February 2025, alongside its classic physics model [10].

For a national meteorological service such as the General Directorate of Meteorology (DGM), this rapid evolution presents a practical engineering challenge. While these models are computationally efficient and widely accessible, their localized accuracy remains uncertain. Their accuracy over Morocco is not well documented, and neither is the extent to which their errors can be corrected. My internship started from these two gaps.

Objectives of the internship

The work had four objectives:

- measure the error of eight AI weather models over Moroccan stations, for five weather variables;
- find what explains this error, for example the hour of the day, the forecast horizon or the location of the station;
- train machine learning models that predict the error and correct the forecast;
- check whether the corrected forecast is really better than the original one, and by how much.

Problem statement

An AI weather model generates a value for a broad grid point, while a weather station measures conditions at a precise location and altitude. Because these two values represent different physical realities, a gap between them is normal. However, this difference is not random. A model might be consistently too warm at night at a specific station, or too cold during the afternoon in mountainous regions. These systematic errors follow detectable

patterns that can be modeled and mitigated. The primary challenge is identifying the exact input features required for the correction model and quantifying the amount of error it can successfully eliminate.

Chapter 1

The host organisation

Outline

1.1 History and missions 12
1.2 Internal organisation 13
1.3 Research and innovation 14
1.4 Project Context 14

As Morocco's national weather service, the DGM provides the institutional context for this work. This chapter presents its main missions, organisation, and role in meteorological research. It then introduces the research subject addressed during the internship and the main question behind it: how AI-based weather forecasts perform when evaluated at observation stations across Morocco.

1.1 History and missions

The General Directorate of Meteorology (DGM) is the public body responsible for meteorology in Morocco. It works under the Ministry of Equipment and Water, and it is known to the public as Maroc Météo. It was created in 1961.

Two major changes shaped the organisation that exists today. The first took place at the beginning of the 1990s, when the directorate became a *Service d'État Géré de Manière Autonome* (SEGMA). This status gave more freedom in the way its services were organised and sold. It allowed the DGM to develop products for economic sectors while keeping its public service role. The second change was the development of regional services. Four regional directorates were created in 2002. In 2016 the network grew to six: Agadir, Beni Mellal, Casablanca, Fès, Rabat and Laâyoune.

The DGM obtained ISO 9001 certification for its quality management system in 2014. A project is also under way to turn the directorate into a public establishment named "Maroc-Météo", with more autonomy in its governance.

The missions of the DGM are the following: observe the atmosphere and the state of the sea, monitor their evolution, maintain the national climatological archive, and provide meteorological and climate services. These missions have one main purpose: protecting people and property. Aviation, agriculture, maritime transport, water management and civil protection all rely on weather forecasts to plan their activities. The strategic vision of the DGM looks towards 2030, with the aim of becoming a reference at the national and regional levels.

1.2 Internal organisation

The DGM has a hierarchical structure. The Director General coordinates the activity and applies the strategic orientations. Below this level, the work is shared between two technical directorates and several support services, as shown in Figure 1.1.

The **Directorate of Forecasting and Meteorological Research** contains the National Forecasting Centre, the National Centre for Meteorological Research and the Information Systems Division. This is where numerical modelling, applied research and the daily forecast production take place. The services cover general forecasting, marine meteorology, aeronautical meteorology, numerical modelling, remote sensing, telecommunications, computing systems and statistics.

The **Directorate of Observation Systems** contains the National Climate Centre and the Technical Affairs and Equipment Division. It manages the surface and upper-air observation networks, the metrology of the instruments, the climatological database and air quality. A third division deals with quality, cooperation and commercialisation, so with the ISO 9001 processes, the scientific partnerships and the sale of meteorological products.

The Directorate of Financial and Administrative Affairs manages human resources, budget and logistics. An audit and management control service reports directly to the Director General. Finally, the six regional directorates bring the services closer to local needs.

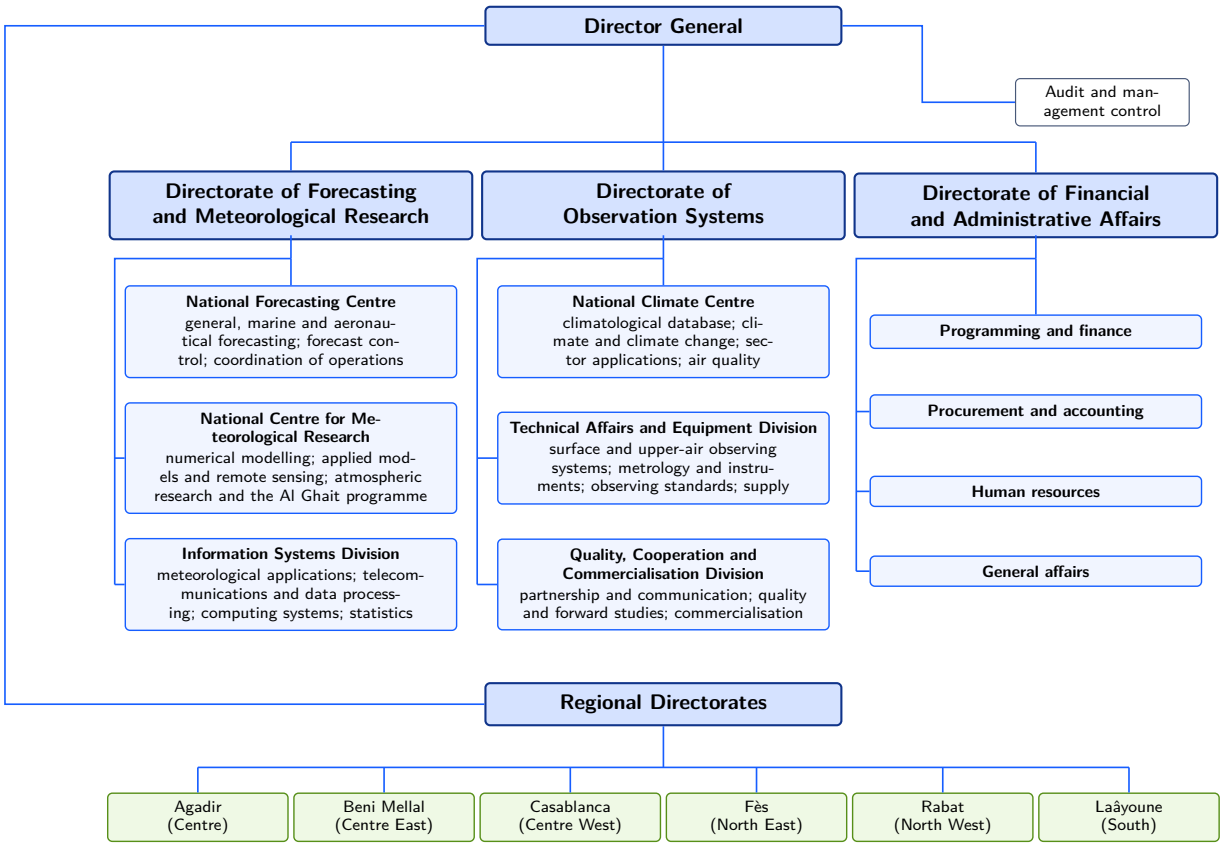


Figure 1.1: *Organisation of the General Directorate of Meteorology. The chart was redrawn from the internal organisation document of the host organisation [18].*

1.3 Research and innovation

Beyond daily forecasting operations, the organization conducts extensive scientific research, which is the specific domain this internship is affiliated with.

Numerical modelling is the visible part: physical equations that describe the atmosphere are solved on a grid, and the models are extended to include new variables and finer resolutions. Less visible, but just as important, is observation. The network is modernised with automatic stations, Doppler weather radars, lightning detection systems and satellite data, because a forecast is only as good as the observations it starts from and is checked against. Alongside these activities is climate research, with studies on long-term trends, extreme events and their effects on water resources and agriculture.

New algorithmic approaches are also part of this work, including probabilistic forecasting and machine learning. The DGM also works with the World Meteorological Organization and foreign research centres, which supports technology transfer.

1.4 Project Context

This internship project is situated directly within an operational framework. While several artificial intelligence weather models are now readily available, a national weather

service must understand how these tools perform over its specific territory before using them in official forecasts. Addressing a practical engineering problem with no predefined solution also reflects the transition from academic coursework to a professional working environment.

The performance metrics published by the developers of these models are typically calculated on large global grids and generally rely on reanalysis data. Consequently, they may not reflect local accuracy across different regional climates and seasonal conditions. A broad global average cannot show how a model will behave at a specific station during a particular period of the year. The primary objective of this work is therefore to bridge this gap by conducting a targeted and localized evaluation for practical application.

Chapter 2

Literature review

Outline

2.1 From physics models to AI weather models 16
2.2 Why a good forecast still needs a correction 17
2.3 Grid resolution and the representativeness error 17
2.4 Statistical post-processing before machine learning 18
2.5 Neural networks in meteorological post-processing 19
2.6 Overconfidence and probability calibration in deep learning 19
2.7 Algorithm choice and the size of the correction dataset 20
2.8 Summary of the background 20

This chapter first introduces AI-based weather forecasting, the sources of systematic errors in weather forecasts, and the statistical methods traditionally used to correct them. It then presents the three research papers that provided the main theoretical basis for the methods used in Chapter 3. Two of these papers come from the machine learning literature and one from meteorology. They were selected because their methods and findings directly influenced the methodological choices made in this work.

2.1 From physics models to AI weather models

A classic weather model solves the equations of fluid dynamics on a three-dimensional grid. While effective, this method is computationally intensive. Executing a global simulation requires supercomputing infrastructure and takes several hours to complete.

A data driven model operates on a fundamentally different paradigm. It is trained on decades of past atmospheric fields, usually the ERA5 reanalysis, and it learns to map the state of the atmosphere at time t to the state at time $t + 6$ hours. Once training is finished, producing a forecast is only a forward pass through the network, so it takes minutes. Table 2.1 lists the eight models present in my dataset.

Table 2.1: *The eight AI weather models studied in this report. The architecture information comes from external sources [14, 13, 12, 15, 16, 17, 11]; the variables come from the dataset I received.*

Model	Developer	Variables available in my dataset
AIFS	ECMWF	t2m, td2m, msl, sp, tp06
Aurora	Microsoft	t2m, msl
DLWP	University of Washington	t2m only
FengWu	Shanghai AI Laboratory	t2m, msl
FourCastNet	NVIDIA	t2m, msl, sp
FuXi	Fudan University	t2m, msl, tp06
GraphCast	Google DeepMind	t2m, msl, tp06
Pangu6	Huawei	t2m, msl

AIFS is the model that went into operations at ECMWF on 25 February 2025, which makes it the first machine learning model used operationally by a major forecasting centre [10]. The others are research models, and most of them publish their forecasts openly.

2.2 Why a good forecast still needs a correction

Even highly accurate models produce systematic errors, primarily due to two distinct factors.

The first one is representativeness. A model gives one value for a grid box of about 25 km. A station measures at one point, at its own altitude and its own environment. Rasp and Lerch give a clear example [1]: if a station sits in a valley and the model grid box represents a higher average altitude, the model will predict colder temperatures than the station measures, every day, in the same direction.

The second reason is the model itself. A learned model tends to produce smooth fields, because smoothing reduces the average error during training. Smooth fields can miss the extreme values.

An error that repeats is called a systematic error, or a bias. It is defined here as

$$\text{bias} = \text{predicted value} - \text{observed value}.$$

A positive bias means the model is too high. Post-processing uses past biases to correct future forecasts. In the terminology of the field, the raw forecast is the input and the corrected forecast is the output.

2.3 Grid resolution and the representativeness error

The models in Table 2.1 are trained on the ERA5 reanalysis at a resolution of 0.25 degrees [6]. A quarter of a degree of latitude is about 28 km on the ground, so one grid cell

covers roughly 28 km by 24 km at the latitude of Morocco. The value that the model gives for that cell is an average over the whole surface, including whatever the cell contains: a plain, a slope, a lake, or a piece of coast.

Conversely, a weather station records strictly localized conditions. It measures the air exactly 2 m above a specific geographical point, capturing the unique altitude and local environment of that exact location. The two numbers answer different questions, so a difference between them is expected even when the model is working perfectly. This difference is called the representativeness error.

It stays small in flat, uniform terrain. Two situations can make it large, and Morocco supplies both.

Altitude

Temperature falls with height at roughly 6.5°C per 1000 m in the free atmosphere. If the average altitude of the grid cell is 300 m below the station, the model will read about 2°C too warm at that station, every day, in the same direction. Morocco provides an optimal geographical context for observing this phenomenon, as the Middle Atlas and the High Atlas sit next to plains and the coast, and a single 28 km cell can contain both. The station results in Section 4.3 are consistent with this: the mountain and inland stations carry the largest errors, although altitude is not isolated there from the other differences between those stations and the coastal ones.

Surface type

A cell that mixes sea and land will produce a value influenced by the sea surface temperature, which changes slowly. A station a few kilometres inland reacts much faster to the sun. The same argument applies to irrigated land next to dry land, or to a city next to countryside.

The dataset gives a direct measure of this geometry: the distance from each station to the grid point used, which ranges from 2.4 km to 65 km (Table 3.1). Section 4.3 shows what that distance does, and what it does not do.

2.4 Statistical post-processing before machine learning

Applying statistical corrections to model outputs is a well established practice in meteorology. The approach known as Model Output Statistics dates back to the early 1970s [4]. The idea is simple: collect pairs of past forecasts and past observations for a station, fit a regression between them, then apply that regression to future forecasts. The regression absorbs whatever the model does wrong in a repeatable way, including the altitude offset described above.

The method used today in most weather services is EMOS, Ensemble Model Output Statistics. It assumes that the temperature follows a normal distribution, and it writes the mean and the spread of that distribution as simple linear functions of the ensemble mean

and the ensemble spread [1]. A version with one set of coefficients per station performs better than a single global version, which is another way of saying that the correction is local.

Machine learning functions within this framework as a highly flexible regression technique. It accommodates numerous inputs and seamlessly models nonlinear relationships. However, the fundamental operational logic remains identical: historical errors are leveraged to correct future forecasts, and the operational value is evaluated directly against the raw predictions.

2.5 Neural networks in meteorological post-processing

Of the three papers reviewed here, *Neural networks for postprocessing ensemble weather forecasts* by Rasp and Lerch [1] is the most closely aligned with my work. Their study investigated whether neural networks could outperform traditional meteorological post-processing by replacing standard linear assumptions with learned, nonlinear relationships. Focusing on 2 m temperatures across 537 German stations, the authors evaluated neural network architectures against quantile regression forests and standard EMOS benchmarks, testing them both globally and on an individual station basis.

To predict the mean and spread of a Gaussian forecast distribution, their networks minimized the CRPS while incorporating additional weather variables and learned station embeddings. This setup achieved a CRPS of 0.78, which represents a 33% improvement over the raw ECMWF ensemble (1.16), demonstrating the clear benefit of auxiliary features and localized spatial representations.

My study draws on three core insights from this work. First, I adopt a learned model to correct systematic forecast errors. Second, while I do not use station embeddings directly, I supply latitude and longitude to provide the network with equivalent spatial context. Finally, I implement early stopping to prevent overfitting, as detailed in Section 3.7.

2.6 Overconfidence and probability calibration in deep learning

The second paper, *On calibration of modern neural networks* by Guo and colleagues [2], shifts focus from meteorology to general machine learning calibration. The study examines whether the probability scores generated by a model accurately reflect their true likelihood of being correct. To quantify the gap between predicted confidence and actual accuracy, the authors utilized the Expected Calibration Error, which groups predictions by confidence levels. They then evaluated various correction methods ranging from a single scaling parameter to highly flexible transformations.

Their findings reveal that modern deep networks are frequently overconfident. A model might predict an outcome with 99% confidence yet be correct far less often than its probability score suggests. The most effective correction proved to be temperature scaling, a

method that divides network outputs by a single learned value prior to the softmax function. Applying this technique to the CIFAR 100 dataset using a ResNet 110 architecture reduced the calibration error dramatically from 16.53% to 1.26%, whereas more complex methods suffered from overfitting and performed poorly.

Because my research involves a regression problem, I did not apply temperature scaling directly. However, the study provides a critical lesson regarding model complexity by demonstrating that the simplest correction often outperforms highly flexible approaches. A parallel observation emerges in Section 3.11 of this report, where tuning on a limited dataset yielded worse outcomes than simply reusing hyperparameters established on a much larger dataset.

2.7 Algorithm choice and the size of the correction dataset

The third paper, *Predicting Good Probabilities with Supervised Learning* by Niculescu Mizil and Caruana [3], is an older study that examines a broad array of machine learning models. It compares ten distinct families of learning algorithms to determine which architectures produce the most reliable probabilities. By training these algorithms on identical problems, the authors evaluated how closely the predicted confidence matched the actual observed frequencies. They then applied two calibration methods to each model: Platt scaling, which utilizes a sigmoid function with two parameters, and isotonic regression, a highly flexible approach that fits a free increasing curve.

The findings indicate that random forests and bagged trees are naturally well calibrated. In contrast, boosted trees tend to shift their probabilities toward the center and consequently require additional calibration. Furthermore, the study demonstrates that the optimal choice between Platt scaling and isotonic regression depends entirely on the volume of available data. Platt scaling proves to be the safer option when the calibration dataset is small, whereas isotonic regression typically requires a threshold of roughly 1,000 examples to become the superior choice.

This insight regarding dataset size and model flexibility is highly relevant to my current work. Because the multiparameter configuration for AIFS contained only 842 training rows, a simpler correction method was necessary to prevent overfitting. This data limitation directly informed and supports the methodological choices detailed in Section 3.11 of this report.

2.8 Summary of the background

The three papers come from different fields and use different data, but they converge on one point: the flexibility of a correction has to match the amount of data available to fit it. Rasp and Lerch reach it from the side of what a large dataset allows, Guo and his colleagues from the side of what a flexible method costs, and Niculescu-Mizil and Caruana

by measuring where the two curves cross. That constraint decided most of the choices made in Chapter 3.

- AI weather models are fast and accurate at global scale, but they are checked against gridded data, not against local stations.
- Local forecasts keep a systematic error, coming from the difference between a grid box and a station, and from the model itself.
- Machine learning can remove a part of this error, and the gain should be measured against the raw forecast.
- Station identity and location carry information that helps the correction.
- A flexible correction model needs a large dataset. With few samples, a simpler choice generalises better.

Chapter 3

Datasets and methodology

Outline

3.1 Description of the data . . . 22
3.2 The five weather variables . . . 25
3.3 The station network . . . 26
3.4 Data preparation . . . 26
3.5 Feature engineering . . . 27
3.6 Forecast times: run time, lead time and valid time . . . 28
3.7 Theoretical background of the four algorithms . . . 29
3.8 Formulas used in the analysis . . . 30
3.9 How the bias was analysed . . . 31
3.10 The calibration method . . . 32
3.11 Hyperparameter tuning . . . 33
3.12 A separate method for precipitation . . . 34
3.13 Evaluation metrics . . . 35

This chapter presents the main steps of the project, starting with data preparation and feature construction, followed by the modelling methods and evaluation of the corrected forecasts. The data, weather variables, stations, and derived features are first described, followed by the four algorithms, their optimisation objectives, and the evaluation metrics.

3.1 Description of the data

My internship supervisor provided the data as a single merged file named `dataset_fusionnée.csv`. Every row in this document pairs a prediction generated by an artificial intelligence model with the actual measurement recorded at a station at that exact moment. This dataset was originally assembled for a broader internal project at the DGM. Therefore, the initial data extraction and merging steps were already completed when I received it.

My own work focuses strictly on analyzing the eight specific models listed in Table 2.1. Although I started working directly from this merged table, I independently verified that the forecasts and observations were matched correctly. The details of these two verification checks are explained in Section 3.6.

Restricted to those eight models the file holds **261,082 rows**. Table 3.1 summarises its content.

Table 3.1: *Main characteristics of the dataset used during the internship.*

Item	Value
Rows (forecast and observation pairs)	261,082
AI weather models	8
Weather stations	34, in Morocco
Weather variables	t2m, td2m, msl, sp, tp06
Period	4 September 2025 to 31 March 2026
Valid times	00, 06, 12 and 18 UTC
Forecast horizon (lead time)	0 to 72 hours
Station to grid point distance	2.4 km to 65.0 km
Missing numerical values	none

Columns

Each row contains the run time and the valid time of the forecast, the lead time in hours, the model name, the station name, the latitude and the longitude, the distance between the station and the grid point used, the variable name, the predicted value, the observed value, and three error columns computed in advance: the bias, the absolute error and the squared error.

Dataset structure

Table 3.2 displays the initial rows of the file prior to any computational steps. The three error columns on the right are included in the file as originally received.

Table 3.2: *First rows of the merged file, for the DLWP model at lead time 0. The columns are shown in two blocks because they do not fit on one line. Values are taken from the file as received.*

run_time	lead_time	model	station	distance_km	lat	lon
2025-09-04	0	dlwp	LAAYOUNE	11.54	27.25	−13.25
2025-09-04	0	dlwp	SIDI-IFNI	14.55	29.25	−10.25
2025-09-04	0	dlwp	TANGER-AERO	8.92	35.75	−6.00
2025-09-04	0	dlwp	LARACHE	14.36	35.25	−6.00
2025-09-04	0	dlwp	EL-HOCEIMA	11.34	35.25	−3.75

variable_name	pred_value	valid_time	obs_value	biais	erreur_abs	erreur_sq
t2m	20.28	2025-09-04 00:00	21.8	−1.52	1.52	2.32
t2m	19.08	2025-09-04 00:00	19.6	−0.52	0.52	0.27
t2m	22.78	2025-09-04 00:00	23.3	−0.52	0.52	0.27
t2m	21.48	2025-09-04 00:00	22.5	−1.02	1.02	1.05
t2m	21.88	2025-09-04 00:00	19.3	+2.58	2.58	6.64

Coverage is not the same for every model

The models do not all publish the same variables, which limits what can be done with each of them. Table 3.3 gives the number of rows per model and per variable. AIFS is the only model with all five variables. DLWP has temperature only, so it cannot be used in the multi-parameter configuration described later.

Table 3.3: *Number of rows available for each AI model and each weather variable. An empty cell means the variable is not available for that model in this dataset.*

Model	msl	sp	t2m	td2m	tp06
AIFS	2,136	2,497	24,611	2,508	23,700
Aurora	1,880		21,474		
DLWP			21,474		
FengWu	1,880		21,474		
FourCastNet	1,880	2,168	21,474		
FuXi	1,880		21,474		20,932
GraphCast	1,880		21,474		20,932
Pangu6	1,880		21,474		

Temperature is by far the best covered variable, and this is why the calibration work focuses on t2m, with precipitation as a second target.

3.2 The five weather variables

The five variables are not interchangeable, and every statistic in this report is computed separately for each specific parameter. Table 3.4 defines each variable, outlines its storage unit, and identifies its primary end users at the DGM.

Table 3.4: *The five weather variables in the dataset. The last column gives the main reason a national weather service produces the variable, which is also the reason a bias in it has a cost.*

Code	Unit	Definition	Who uses it
t2m	°C	Air temperature measured in a shelter 2 m above the ground	Everyone: public bulletins, agriculture, energy demand, health warnings
td2m	°C	Temperature at which the air would become saturated; with t2m it gives the humidity	Aviation, for fog and visibility
msl	hPa	Surface pressure reduced to sea level, so that stations at different altitudes compare	Synoptic analysis, storm tracking
sp	hPa	Pressure at the real altitude of the station, with no reduction	Altimetry, and any calculation that needs the true air density
tp06	mm	Depth of water accumulated during the six hours ending at the valid time	Water management, flood warning, agriculture

The units are critical for correctly interpreting the tables in Chapter 4. An error of 2°C on t2m and an error of 2 hPa on msl represent fundamentally different physical quantities, and averaging them together would produce a mathematically meaningless result. The same restriction applies within a single column of the raw file, which necessitates grouping the data by variable prior to any statistical computation.

Surface pressure and precipitation require specific contextualization before examining the final results. Surface pressure is the variable most directly impacted by station altitude, which provides the primary explanation in Section 4.1 for the exceptionally large AIFS error on that specific parameter. Furthermore, precipitation is the only variable whose distribution is overwhelmingly dominated by zeros. This unique statistical distribution completely alters the required methodology, which is addressed independently in Section 3.12.

3.3 The station network

The 34 stations belong to the Moroccan surface network and report at the synoptic hours 00, 06, 12 and 18 UTC. They cover the country from Tangier to Dakhla, and they sit in places that are physically very different from each other. Table 3.5 lists them in three groups.

The stations are grouped into three topographic families for this analysis. A station beside the Atlantic, a station on an inland plain and a station on a plateau at altitude do not present the same problem to a model whose cell is about 28 km wide.

Table 3.5: *The 34 stations of the dataset, grouped by the kind of place they sit in. The grouping is used in Section 4.3 to read the per-station errors.*

Group	Stations	Count
Coastal	Agadir, Ait Melloul, Casa-Anfa, Dakhla, El Hoceima, El Jadida, Essaouira-Port, Kenitra, Laâyoune, Larache, Mohammedia, Rabat-Salé, Safi, Sidi Ifni, Tanger-Aéro, Tan-Tan, Tétouan	17
Inland plains and plateaus	Bouarfa, Fès-Saïss, Guelmim, Khouribga-Aéro, Marrakech, Meknès, Nouasseur, Oujda, Settat, Sidi Slimane, Smara, Taroudant, Tiznit	13
Mountain and pre-Saharan	Errachidia, Ifrane, Midelt, Ouarzazate	4

The network distribution is intentionally asymmetric, prioritizing population centers and airports over uniform topographic coverage. Half of the stations are located near the coast, while only four are situated at high altitudes or beyond the Atlas Mountains. This geographical imbalance significantly impacts the station specific error results detailed in Section 4.3, and it must be considered within the study limitations, as a correction model trained predominantly on coastal stations processes far fewer examples of the mountainous regime.

3.4 Data preparation

Data preparation was straightforward as the provided file required minimal cleaning. I converted the valid time column from text to a date and time format, then extracted the hour and the month. There were no missing numerical values to handle.

A critical consideration during this phase was the handling of physical units. The summary statistics of the predicted and observed columns mix several physical units in the same column: degrees Celsius for temperature, hectopascals for pressure, millimetres for rain. A global mean over that column has no physical meaning. Every statistic in this report is therefore computed per variable, and most of them per model as well.

3.5 Feature engineering

The exploratory analysis showed that the bias changes during the day, so the exact hour must be provided to the calibration models. Using the raw numerical values is ineffective because time is circular. Hour 18 and hour 0 are exactly six hours apart, but their numerical values suggest a large gap. To capture the cyclical nature of time, I applied a trigonometric encoding to the valid hour:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot \text{hour}}{24}\right), \qquad \text{hour_cos} = \cos\left(\frac{2\pi \cdot \text{hour}}{24}\right).$$

I then added two binary features based on what the data showed, and not on a general rule:

- `is_night`, equal to 1 at hours 0 and 6, because these two hours behave in a similar way;
- `is_afternoon`, equal to 1 at hour 12, because the midday bias follows a clearly different pattern.

Hour 18 was not assigned an independent binary feature because its bias remains within the normal statistical range. Creating a dedicated column for this specific time would merely introduce mathematical noise.

While the cyclical pair (`hour_sin`, `hour_cos`) uniquely identifies all four valid hours, the supplementary binary features optimize the data representation for specific algorithms. A decision tree requires only a single split on `is_night`, whereas extracting the identical regime from trigonometric values demands multiple splits. Similarly, neural networks can directly weight this binary indicator rather than approximating complex threshold boundaries. Section 4.7 evaluates the impact of this structural optimization on the tree based models.

The full list of inputs

Table 3.6 gives every feature that reaches a calibration model, with its unit and the configuration it belongs to. This input list is intentionally concise. Seven of the eight Config A features mathematically describe the geographic and temporal context, while the remaining feature provides the raw forecast value.

Table 3.6: *Input features of the calibration models. Config A uses the first block only. Config B adds the other variables predicted by the same AI model, so its exact list depends on the model (Table 3.3).*

Feature	Meaning	Unit or range	Config
`pred_value`	Value forecast by the AI model for the target variable	°C for t2m	A and B
`lat`	Latitude of the station	degrees north	A and B
`lon`	Longitude of the station	degrees east	A and B
`lead_time`	Hours between the run time and the valid time	0 to 72 h	A and B
`hour_sin`	Sine of the valid hour on a 24 h circle	−1 to 1	A and B
`hour_cos`	Cosine of the valid hour on a 24 h circle	−1 to 1	A and B
`is_night`	1 at 00 and 06 UTC, 0 otherwise	0 or 1	A and B
`is_afternoon`	1 at 12 UTC, 0 otherwise	0 or 1	A and B
`pred_msl`, `pred_sp`, `pred_td2m`, `pred_tp06`	Values forecast by the same AI model for its other variables, at the same station, run and lead time	hPa, hPa, °C, mm	B only

One rule applies to the whole table. Every feature in it is known at the moment the forecast is issued, and the observation never appears among the inputs, in any configuration, for any target. A model that received the observed temperature as an input would predict the bias almost perfectly and would be worthless in operations, because at forecast time the observation does not exist yet. The same rule is what forced the two-step design for precipitation in Section 3.12: the label of step 1 is derived from the observation, so it can be used as a target but never as a feature.

3.6 Forecast times: run time, lead time and valid time

Three distinct temporal labels structure the dataset: the *run time* (model initialization), the *lead time* (forecast horizon in hours), and the *valid time* (the actual target moment the forecast describes). Their relationship is defined as:

$$\text{valid time} = \text{run time} + \text{lead time}.$$

To ensure evaluation accuracy, predictions must be paired with observations recorded exactly at the valid time. The provided dataset successfully implements this alignment. Furthermore, all engineered temporal features, such as `hour_sin` and `is_night`, are derived exclusively from the valid time. This distinction is critical because the physical phenomena driving systematic bias depend on the specific time of day the forecast represents rather than the model initialization time.

3.7 Theoretical background of the four algorithms

The four selected algorithms represent distinct machine learning families. This section outlines the theoretical foundation and optimization objective for each algorithm to provide context for the results presented in Chapter 4.

All four solve the same supervised problem. Given n training examples $(\mathbf{x}_i, y_i)$, where $\mathbf{x}_i$ holds the features of one forecast and y_i is the bias of that forecast, we look for a function f such that $f(\mathbf{x}) \approx y$ on data the model has not seen.

Multi-layer perceptron

An MLP is a stack of layers. Each layer multiplies its input by a matrix of weights, adds a vector of biases, and applies a nonlinear function:

$$\mathbf{h}^{(l)} = \varphi\Big(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big), \qquad \mathbf{h}^{(0)} = \mathbf{x}.$$

The network used here has two hidden layers of 64 and 32 units, with $\varphi = \tanh$, and a single output unit with no activation, since the target is a real number. The weights are chosen to minimise the mean squared error

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}\Big(y_i - f(\mathbf{x}_i)\Big)^2,$$

using the Adam optimiser, a form of gradient descent with an adaptive step size. The nonlinearity is what lets the network represent a smooth curve between the hour of the day and the bias, which is the reason it does well on the three time-driven models.

Two practical considerations emerge from this mathematical formulation. Input features require standardization because variables with large numerical ranges, such as the forecast lead time, would otherwise improperly dominate bounded trigonometric variables like the sine and cosine of the hour. Furthermore, the training process must be explicitly halted when the validation error plateaus, which justifies the implementation of an early stopping mechanism.

Support vector regression

SVR does not try to pass through every point. It looks for a function that stays within a tube of width ε around the data, and it only pays a penalty for the points outside the tube. With a kernel, the prediction is written as a weighted sum of similarities to the training points:

$$f(\mathbf{x}) = \sum_{i=1}^{n}(\alpha_i - \alpha_i^*)\,K(\mathbf{x}_i, \mathbf{x}) + b, \qquad K(\mathbf{u}, \mathbf{v}) = \exp\Big(-\gamma\|\mathbf{u} - \mathbf{v}\|^2\Big).$$

The parameter C sets how strongly the points outside the tube are penalised, ε sets the width of the tube, and γ sets how far the influence of one training point reaches. Like the MLP, SVR compares distances between examples, so it also needs standardised inputs.

The ε parameter defines a margin of tolerance where errors incur no mathematical penalty, which dictates how tightly the model fits small fluctuations.

Random forest

A random forest algorithm constructs an ensemble of decision trees and aggregates their predictions [8]:

$$f(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x}).$$

Each tree is grown on a bootstrap sample of the rows, and at each split only a random subset of the features is considered. Both sources of randomness force the trees to differentiate from each other, and averaging these distinct trees reduces the prediction variance without significantly increasing its bias. A single deep tree would fit the training set almost perfectly and generalize poorly; the aggregated average of many such trees mitigates this risk.

Because decision trees partition data based on numerical thresholds, feature scaling is irrelevant, entirely eliminating the need for standardization. Furthermore, they natively process binary features such as `is_night` by utilizing them as direct split criteria, an attribute highly relevant to the feature importance analysis detailed in Section 4.7.

Gradient boosting with LightGBM

Gradient boosting also utilizes decision trees, but it constructs them sequentially, with each subsequent tree explicitly trained to correct the residual errors of the preceding ensemble:

$$f_K(\mathbf{x}) = \sum_{k=1}^{K} \eta\, T_k(\mathbf{x}),$$

where η is the learning rate. At step k the new tree is fitted to reduce the current loss, and the objective includes a penalty on the complexity of the tree:

$$\mathcal{L}^{(k)} = \sum_{i=1}^{n} \ell\Big(y_i,\ f_{k-1}(\mathbf{x}_i) + T_k(\mathbf{x}_i)\Big) + \Omega(T_k), \qquad \Omega(T) = \gamma L + \tfrac{1}{2}\lambda \sum_{j=1}^{L} w_j^2,$$

with L the number of leaves and w_j the value in leaf j. LightGBM implements this with two choices that make it fast on tabular data: features are grouped into histogram bins before searching for a split, and trees grow leaf by leaf, always splitting the leaf that promises the largest gain [7].

The fundamental algorithmic distinction between growing trees independently and sequentially significantly impacts the results in Section 4.7. The two methodologies generate divergent feature importance rankings precisely due to these contrasting split selection mechanisms.

3.8 Formulas used in the analysis

Error metrics

With y_i the true value and $\hat{y}_i$ the predicted one:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Because RMSE squares the individual errors, outlier deviations disproportionately influence the final metric. This mathematical property establishes RMSE as the standard reference score in meteorology, as a single forecast deviation of 10°C generates more severe operational consequences than ten separate deviations of 1°C. Furthermore, the R^2 metric evaluates the model against a constant baseline prediction equal to the dataset mean. Consequently, a negative R^2 value indicates that the predictive model performs mathematically worse than this constant baseline.

Analysis of variance

The ANOVA test used in Section 4.2 asks whether the mean bias is the same in every part of the day. The null hypothesis is that the four group means are equal. The statistic compares the variance between the groups with the variance inside them:

$$F = \frac{\text{variance between groups}}{\text{variance within groups}} = \frac{\sum_g n_g(\bar{y}_g - \bar{y})^2/(k-1)}{\sum_g \sum_{i \in g}(y_i - \bar{y}_g)^2/(n-k)}.$$

If the time of day exerted no influence, the F statistic would approximate 1. The p value represents the probability of observing this F statistic assuming the group means are genuinely equal. Given the massive sample size exceeding 21,000 rows per model, the p value approaches zero at the slightest deviation; therefore, the true analytical value lies in the magnitude of the F statistic rather than the p value itself.

Mutual information

Mutual information measures how much knowing a feature X reduces the uncertainty about the bias Y:

$$I(X;Y) = \iint p(x,y)\, \log \frac{p(x,y)}{p(x)\,p(y)}\, \mathrm{d}x\, \mathrm{d}y.$$

Mutual information equals zero for strictly independent variables and increases proportionally with the strength of their association. Unlike the Pearson correlation coefficient, it does not assume a linear relationship. This property is crucial for this analysis because the correlation between the valid hour and the bias follows a nonlinear curve. Calculating a standard linear correlation would yield a value near zero, entirely masking the underlying physical effect.

3.9 How the bias was analysed

The primary objective of the exploratory analysis was to rigorously characterize the error structure prior to applying calibration models. This phase utilized three distinct analytical approaches.

Error metrics

For every model and every variable I computed the mean bias, the mean absolute error (MAE) and the root mean square error (RMSE). Evaluating the mean bias in isolation is mathematically insufficient because positive morning errors and negative midday errors completely cancel each other out. MAE and RMSE are therefore required to quantify the absolute magnitude of the error independently of its direction.

Aggregations

The mean bias was computed by hour of the day, by date, by month, by year, by lead time, by station, and by temperature range. These aggregations were explicitly designed to determine whether the forecast error is mathematically dependent on these specific parameters.

Statistical checks

Two statistical tests were implemented to mathematically validate the visual patterns observed in the plotted data. A one way ANOVA checked whether the bias fundamentally differs between the four moments of the day, for each model. Mutual information estimated how much predictive information each candidate feature (lead time, hour, month, station, latitude, longitude) carries regarding the bias.

3.10 The calibration method

The correction follows a two-stage logic:

$$\text{corrected forecast} = \text{predicted value} - \text{predicted bias}.$$

A machine learning model is trained to predict the bias from the information known at forecast time. That predicted bias is then subtracted from the raw forecast. The model never sees the observation at prediction time, which is what makes the method usable in real conditions.

Four algorithms

I trained four algorithms, chosen so that different families are represented:

- **MLP**, a small neural network able to learn nonlinear relations;
- **SVR**, a support vector regression with an RBF kernel;
- **Random Forest**, an ensemble of independent decision trees;
- **LightGBM**, a gradient boosting method where each new tree corrects the errors of the previous ones.

MLP and SVR need standardised inputs, because both are sensitive to the scale of the features. I standardised the inputs and the target for these two, and inverted the

transformation on the predictions. The scaler was fitted on the training rows only and then applied to the test rows, so no information from the test period enters the transformation. The two tree-based methods work on the raw values.

Two configurations

Config A (single variable) uses the predicted value of the target variable plus features that are not meteorological: latitude, longitude, lead time and the four time features. The classic definition of single-variable calibration uses only the predicted target, but the exploratory analysis had already shown that location, forecast horizon and hour matter, so leaving them out would have thrown away known information.

Config B (multi-parameter) adds the other weather variables predicted by the same model. For AIFS, for example, the temperature bias is predicted using the predicted temperature, pressure, dew point and precipitation together. Building Config B requires a wide table where one row holds every variable for the same station, run and lead time. The file itself has no missing numerical values, but gaps appear at this stage, because a run may carry one variable and not another at the same lead time. These gaps were filled with the nearest available lead time, and rows that were still incomplete were dropped. This is why Config B always has fewer samples than Config A.

Train and test split

The data was split by time and never shuffled: the oldest 80% for training, the most recent 20% for testing. A random split would let the model learn from days that come after the days it is tested on, which never happens in real forecasting and gives results that are too optimistic.

For temperature in Config A this gives 19,688 training rows and 4,923 test rows for AIFS, and 17,179 training rows and 4,295 test rows for each of the seven other models. The split is applied after sorting on the valid time, and it is applied inside each model and each variable separately, so every experiment is judged on its own most recent period rather than on a period borrowed from another model. The function that performs the split is given in Listing 1, Appendix A.3.

3.11 Hyperparameter tuning

Each algorithm was tuned with Optuna [9], which uses the results of the previous trials to decide where to search next. The score was the negative mean squared error, computed with a 3-fold `TimeSeriesSplit` so that every validation fold comes after its training fold. The budget was 30 trials for the MLP, 25 for the SVR and 40 each for Random Forest and LightGBM. The search ran on AIFS temperature, the largest subset with 19,688 training rows.

Optimizing hyperparameters on a single model necessitates verifying whether these parameters generalize to the other datasets. To evaluate this, I compared the performance of the tuned parameters against the default configurations on DLWP, GraphCast and

FengWu. Table 3.7 gives the change in RMSE. Random Forest gains around 10% on all three models. The MLP gains a little. SVR and LightGBM are close to their defaults, and sometimes slightly worse.

Table 3.7: *Change in RMSE when the hyperparameters tuned on AIFS are applied to three other models. A positive value means the tuned version is better than the default one.*

Algorithm	DLWP	GraphCast	FengWu
MLP	+1.3%	+1.1%	+1.4%
SVR	−1.4%	+0.8%	−2.8%
Random Forest	+11.2%	+9.2%	+10.4%
LightGBM	−0.5%	−0.3%	+1.0%

Tuning on the smaller Config B samples

Config B was also tuned on its own data. For AIFS this table has only 842 training rows against about 20,000 for Config A. I compared three options on the seven models that have a Config B: plain defaults, the Config B search result, and the Config A values reused.

The Config B search underperformed compared to the reused Config A values for SVR on 5 out of 7 models, and for LightGBM on all 7 models. It only outperformed the baseline for Random Forest on 4 out of 7 models. Consequently, I retained the specific Config B optimized values exclusively for Random Forest, while reusing the broader Config A parameters for all other algorithms. Achieving a superior optimization score during a search on a highly restricted sample does not guarantee operational success on unseen data. Over optimizing on a small dataset often degrades overall performance by forcing the model to fit localized statistical noise.

3.12 A separate method for precipitation

Precipitation data required a distinct methodology. More than 98% of the observations record zero rainfall, and only a tiny fraction of the rows show a bias larger than 0.1 mm. A regression model trained directly on this data simply learns the easiest path, which is to always predict a bias of zero. While the overall mean error of such a model appears mathematically low, the resulting correction provides no practical utility because it fails to adjust the forecasts during actual precipitation events.

I used a two-step approach:

1. **Classification.** Predict whether the precipitation bias is significant, defined as an absolute bias above 0.1 mm.
2. **Regression.** For the cases classified as significant, predict the value of the bias.

The class imbalance was handled inside the first step. SVC, Random Forest and LightGBM accept sample weights, so the rare class received a weight equal to the ratio between

the two classes. The scikit-learn MLP classifier does not accept sample weights, so the minority class was oversampled in the training set instead. The test set was left untouched, which means the reported scores describe performance on the real distribution of rainfall.

3.13 Evaluation metrics

A calibration model can be evaluated using two distinct approaches. Separating these metrics is essential to distinguish between a model that merely fits the data mathematically and one that provides practical operational value.

Level 1: quality of the predicted bias

MAE, RMSE and R^2 between the true bias and the predicted bias. R^2 tells how much of the bias variation the model explains. A value of 0 means the model does no better than predicting the mean, and a negative value means it does worse.

Level 2: Effect on the forecast error

This is the definitive operational metric for a meteorological forecaster. For every experiment I computed, on the same test rows, the error of the raw forecast against the observation, then the error of the corrected forecast against the same observation, and finally the improvement:

$$\text{RMSE improvement} = \left(1 - \frac{\text{RMSE}_{\text{corrected}}}{\text{RMSE}_{\text{raw}}}\right) \times 100.$$

A positive value means the correction helped. A negative value means it made things worse, which can happen when the raw forecast was already close to the observation. Listing 2, Appendix A.3, gives the function that computes this.

Purpose of the two levels. A calibration model can have a good R^2 and still be useless, and it can have a modest R^2 and still reduce the forecast error a lot. The second level is the one I use in Chapter 4 to compare the models.

Scores for the precipitation classifier

Step 1 of the precipitation method answers a yes or no question, so the metrics above do not apply to it. Every test row falls into one of four boxes, and the four counts are usually written as a contingency table (Table 3.8).

Table 3.8: *Contingency table for a yes or no forecast. The event here is "the precipitation bias of this row is larger than 0.1 mm in absolute value".*

	Predicted: event	Predicted: no event
Observed: event	TP (hit)	FN (miss)
Observed: no event	FP (false alarm)	TN (correct rejection)

Three scores are built from these counts:

$$\text{Precision} = \frac{TP}{TP + FP} = 1 - \text{FAR},$$

$$\text{Recall} = \frac{TP}{TP + FN} = \text{POD}, \qquad F_1 = 2\,\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Precision and recall carry different names in meteorological verification, and those names say more clearly what they measure. Recall is the probability of detection (POD): among the events that really happened, how many did the system catch. Precision is one minus the false alarm ratio (FAR): among the alarms the system raised, how many were real. The two have to be read together, because either one can be bought at the price of the other. A system that raises an alarm on every single row has a POD of 1 and a FAR equal to one minus the frequency of the event, which for tp06 would be about 0.99. The F_1 score is the harmonic mean of the two and gives one number when a compromise is needed.

Chapter 4

Results

Outline

4.1 Baseline accuracy of the raw AI forecasts . . . 37
4.2 The bias changes with the hour of the day . . . 39
4.3 The bias against lead time, temperature and location . . . 41
4.4 Features that explain the bias . . . 45
4.5 Calibration results for temperature . . . 45
4.6 Precipitation . . . 47
4.7 Consistency between the learned models and the exploratory analysis . . . 48
4.8 Summary of the results . . . 50
4.9 Comparison with published results . . . 50
4.10 Limitations . . . 51

The results are presented in a logical progression. The chapter first evaluates the baseline accuracy of the raw artificial intelligence forecasts over Morocco, then identifies the primary drivers of these errors, and finally quantifies the error reduction achieved by the calibration models. This final assessment directly addresses the practical needs of meteorological forecasters.

4.1 Baseline accuracy of the raw AI forecasts

Table 4.1 gives the error of the eight models for 2 m temperature, computed on all 21,474 to 24,611 pairs available per model.

Table 4.1: *Error of the raw temperature (t2m) forecasts, all stations and all lead times together. Sorted by RMSE.*

Model	Mean bias (°C)	MAE (°C)	RMSE (°C)	Pairs
Aurora	+0.18	1.61	2.12	21,474
FengWu	+0.09	1.64	2.14	21,474
Pangu6	+0.47	1.67	2.19	21,474
FuXi	+0.48	1.71	2.23	21,474
FourCastNet	+0.40	1.80	2.34	21,474
GraphCast	+0.47	3.55	4.51	21,474
DLWP	+2.57	3.94	5.08	21,474
AIFS	−0.15	3.94	5.13	24,611

The models generally cluster into two categories. Five models maintain an RMSE between 2.1°C and 2.35°C. The remaining three perform significantly worse, exhibiting errors between 4.5°C and 5.2°C.

The AIFS results require careful interpretation. Its overall mean bias is exceptionally close to zero, yet it registers the largest RMSE among all eight models. This provides the clearest demonstration in the dataset as to why mean bias is an insufficient metric when analyzed in isolation. The substantial positive errors and negative errors generated by AIFS effectively cancel each other out in the global average. DLWP is the opposite case, with a clear warm bias of +2.57°C that is visible in the mean. Figure 4.1 shows the two behaviours side by side: the AIFS histogram is centred on zero but very wide, while Aurora and FengWu are narrow, and DLWP is shifted to the right of the zero line.

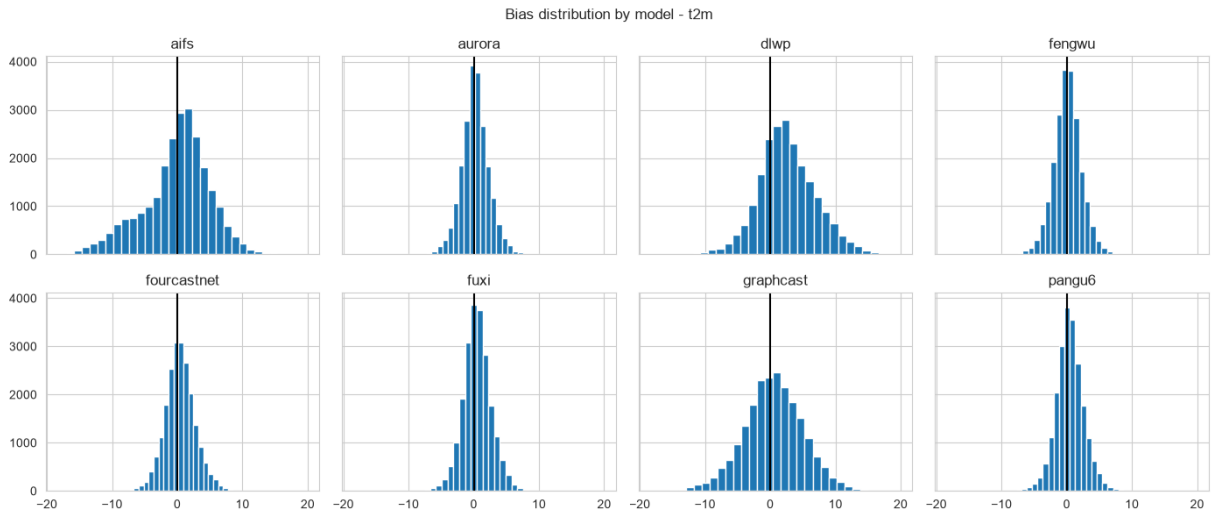


Figure 4.1: *Distribution of temperature bias for each AI model. The vertical line marks zero bias. Histograms with different spreads can have the same mean bias but different RMSE values.*

Figure 4.2 shows the same ranking for all five variables. Mean sea level pressure follows a similar pattern, with FengWu and FuXi around 1.3 hPa of RMSE and GraphCast at 4.9 hPa. Surface pressure is available for two models only, and the very large error of AIFS (RMSE of 32.7 hPa) most likely comes from the altitude difference between the station and the grid point rather than from the model itself. Precipitation shows a low RMSE for all three models that provide it, ranging from roughly 0.47 mm to 0.51 mm. However, this metric does not accurately reflect the actual performance of the models due to the heavy imbalance of zero values, a phenomenon detailed comprehensively in Section 4.6.

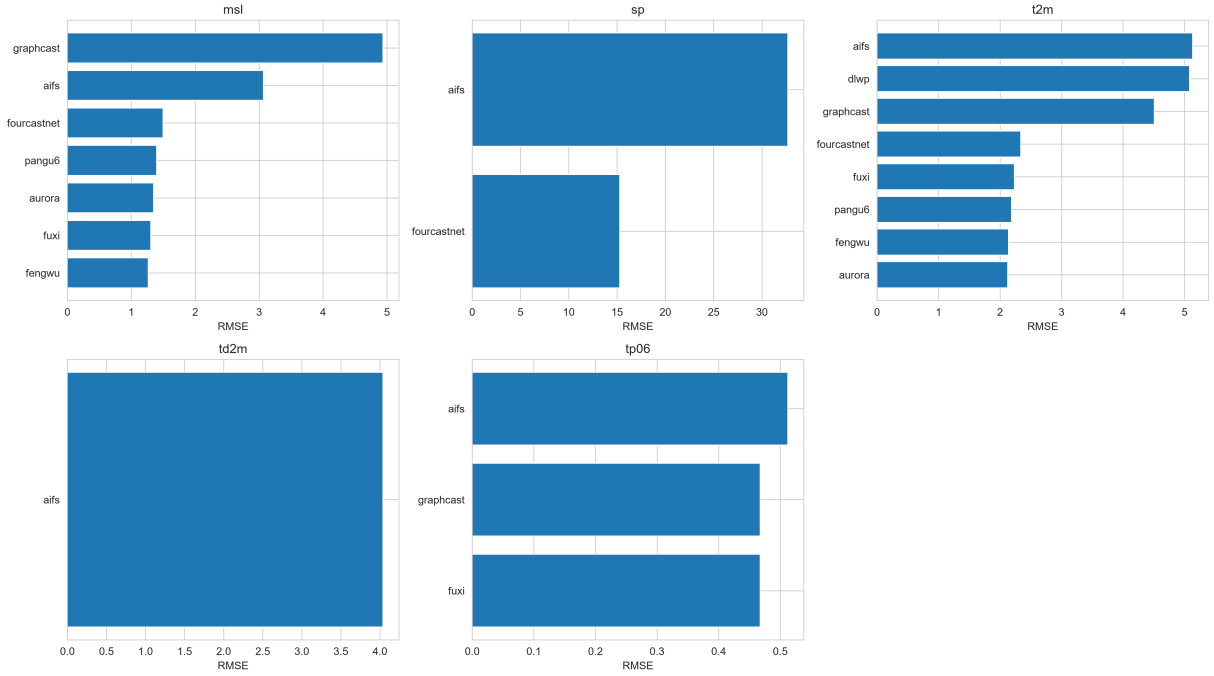


Figure 4.2: *Models ranked by RMSE for each weather variable. Lower is better.*

4.2 The bias changes with the hour of the day

This daily fluctuation constitutes the primary finding of the exploratory analysis. As illustrated in Figure 4.3, five models maintain a near zero bias across all hours, whereas the remaining three exhibit severe temporal deviations.

AIFS goes from about +3.4°C at night to −6.4°C at noon. GraphCast and DLWP show a similar shape with a smaller amplitude. The boxplots by moment of the day in Figure 4.4 give the same message: AIFS has its largest negative bias at noon (−6.35°C), while DLWP is too warm at night (+5.77°C) and at dawn (+4.69°C) and behaves better in the afternoon and the evening.

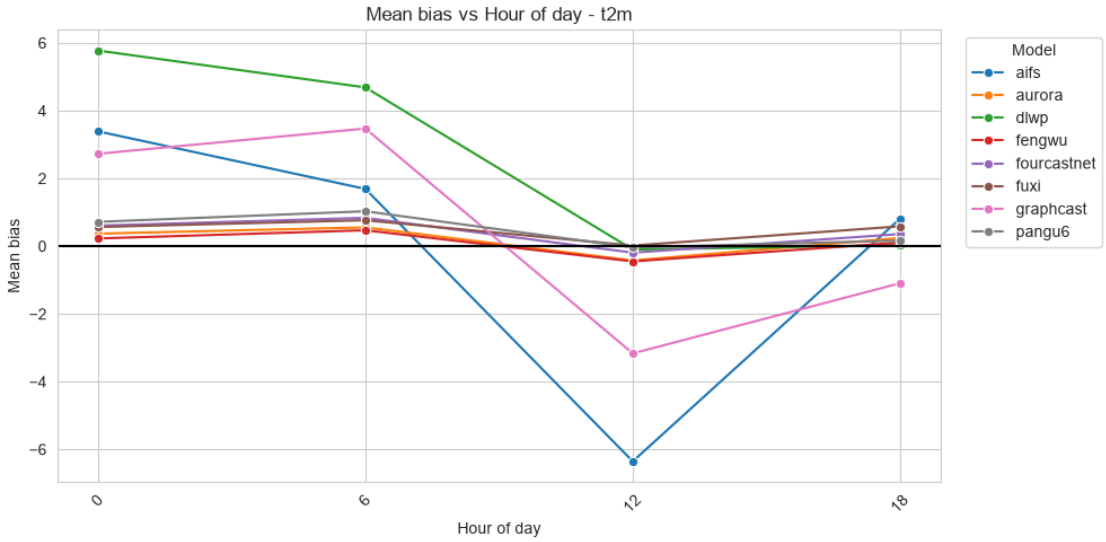


Figure 4.3: *Mean temperature bias against the hour of the day, for the eight models.*

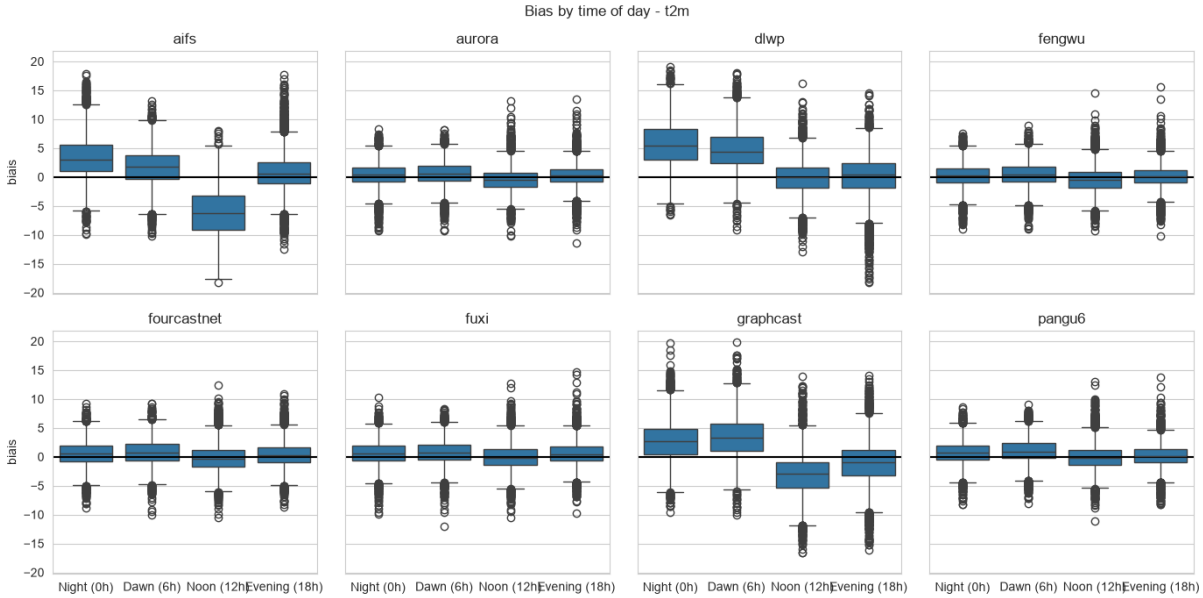


Figure 4.4: *Distribution of the temperature bias by moment of the day, one panel per model.*

An ANOVA test confirms that this daily cycle is statistically significant. As anticipated from the large sample size discussed in Section 3.8, the true analytical value lies in the F statistic: 9072 for AIFS, 4212 for GraphCast and 4160 for DLWP, against 121 to 297 for the five other models. The error of these three models is not random noise: it is organised by the time of day.

Why the error follows the sun

While statistical tests confirm this daily cycle, the underlying cause is physical. Understanding this mechanism is vital to determine if the learned corrections generalize across seasons and geographies. Near the ground, temperature is governed by heat exchange following a strict daily rhythm, which coarse 28 km grid models fail to capture accurately.

During the day

Intense solar heating creates a deep convective boundary layer. Because coarse models overestimate this vertical mixing over broad areas, they dilute the concentrated surface heat, resulting in midday forecasts that are artificially cold.

During the night

Rapid surface cooling creates a shallow stable layer highly sensitive to local topography. Coarse models continuously overmix this shallow layer, erroneously drawing warmer air downward and producing nocturnal forecasts that are artificially warm.

The combined effect is a heavily compressed daily temperature range. While this smoothing is a known limitation of traditional physics models, AI architectures amplify it significantly. AIFS, trained on physics generated reanalysis data, inherits and exacerbates these weaknesses through error minimization algorithms, swinging from a nocturnal overestimation of +3.4°C to a midday underestimation of negative 6.4°C.

This physical mechanism explains the temperature range compression shown in Figure 4.6 and the severe high altitude errors detailed in Section 4.3. Ultimately, the temporal features engineered in Chapter 3 succeed because they mathematically proxy these distinct physical regimes rather than the weather itself.

4.3 The bias against lead time, temperature and location

Lead time

While forecast errors are generally expected to grow continuously with the forecast horizon, the data reveals a different pattern. As illustrated in Figure 4.5, the bias for most variables oscillates with a 24 hour period rather than increasing steadily. The daily cycle dominates the effect of the horizon over the range studied here (0 to 72 hours). Only precipitation shows a slow increase after 24 hours.

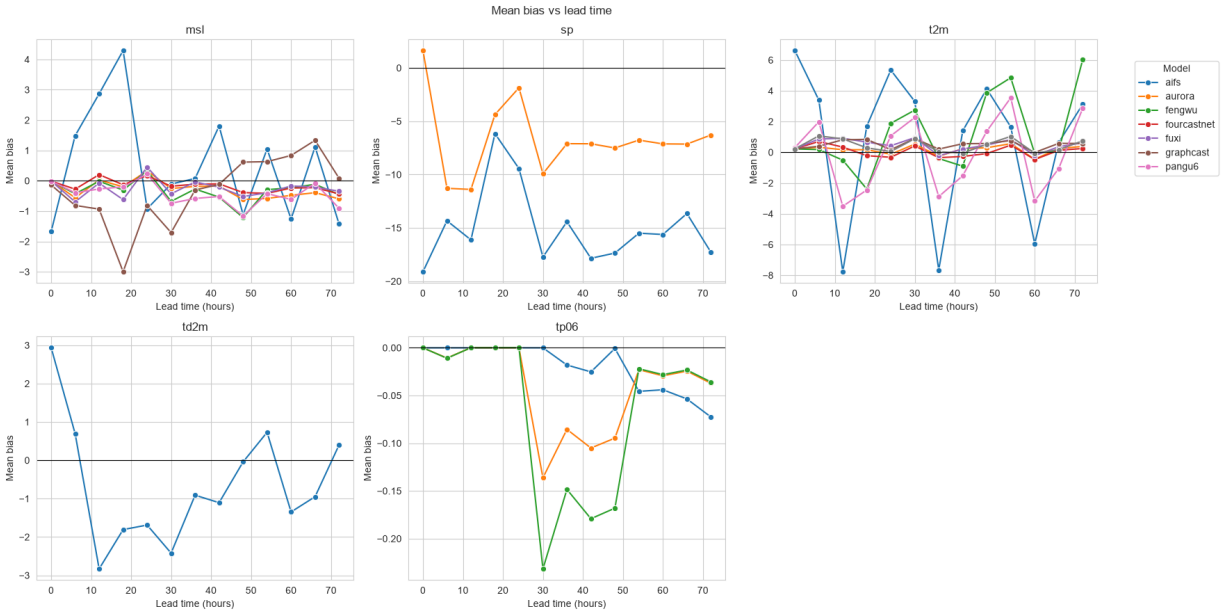
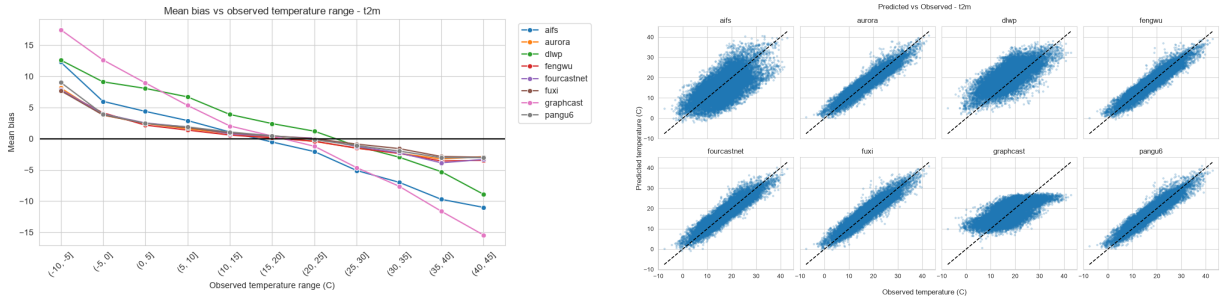


Figure 4.5: *Mean bias against the forecast horizon, for the five weather variables.*

Temperature range

When the observed temperature is split into 5°C bands, all eight models behave in the same way: they overestimate cold temperatures and underestimate warm ones (Figure 4.6). The models are most accurate between 10°C and 25°C. This is the smoothing effect mentioned in Chapter 2. The clearest example is GraphCast, which almost never predicts above 28°C while the observations reach 42.7°C.



(a) Mean bias per observed temperature band (b) Predicted against observed temperature

Figure 4.6: *Two views of the same problem: the models compress the range of temperatures, so cold days are predicted too warm and hot days too cold.*

Station location

Forecast error heavily depends on station topography. As illustrated in Figure 4.7, Ifrane records the highest network RMSE at 5.55°C, while Mohammedia records the lowest at 2.11°C. The fourteen most accurate stations are exclusively coastal, whereas all mountain and pre Saharan stations rank poorly. Ultimately, the exact geographic composition within the 28 km grid cell dictates performance.

This spatial discrepancy aligns with the representativeness error established in Section 2.3. A grid cell over a coastal station generally encapsulates flat terrain thermally regulated by the ocean. Conversely, the equivalent cell over Ifrane mathematically averages a complex topography of plateaus and valleys. Consequently, the coarse global model outputs an aggregated abstraction that inherently fails to capture the highly specific microclimate of the physical observation station.

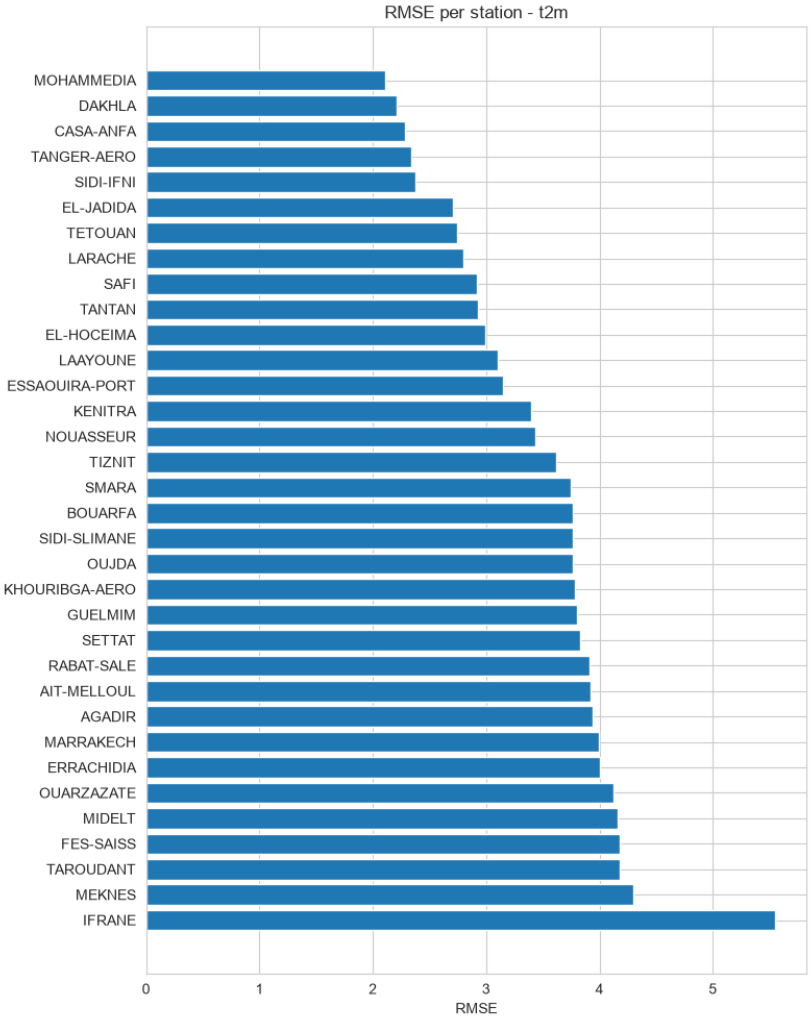


Figure 4.7: *Temperature RMSE per station, all eight AI models and all lead times together, sorted from the easiest station to the hardest one.*

Month

The bias also moves through the period (Figure 4.8), and the two models that move the most are among the three that need the correction the most. DLWP is 3.84°C too warm in November and only 0.63°C too warm in January. GraphCast changes sign inside the period: 1.44°C too warm in December, 1.28°C too cold in March. The five accurate models stay between −0.10°C and +0.91°C in every month of the record. Consequently, a correction model optimized strictly on seven months of data that excludes the summer season captures only a partial representation of the annual error cycle, a limitation explicitly addressed in Section 4.10.

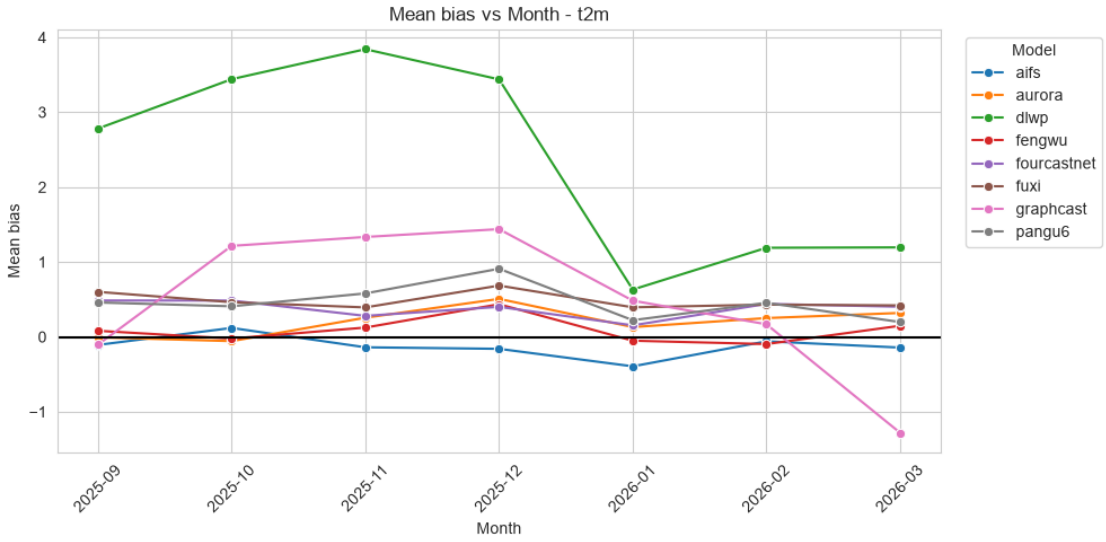


Figure 4.8: *Mean temperature bias per month, for the eight models. The record covers September 2025 to March 2026 and contains no summer month.*

Distance to the grid point

While the overall correlation between the station distance and the absolute bias initially appears significant, calculating this metric within individual models yields near zero values. The global correlation is a statistical artifact driven by model clustering, as demonstrated in Figure 4.9. DLWP deviates entirely from this trend. This indicates that the choice of model architecture exerts a vastly greater influence on the resulting error than the geographic distance to the grid point.

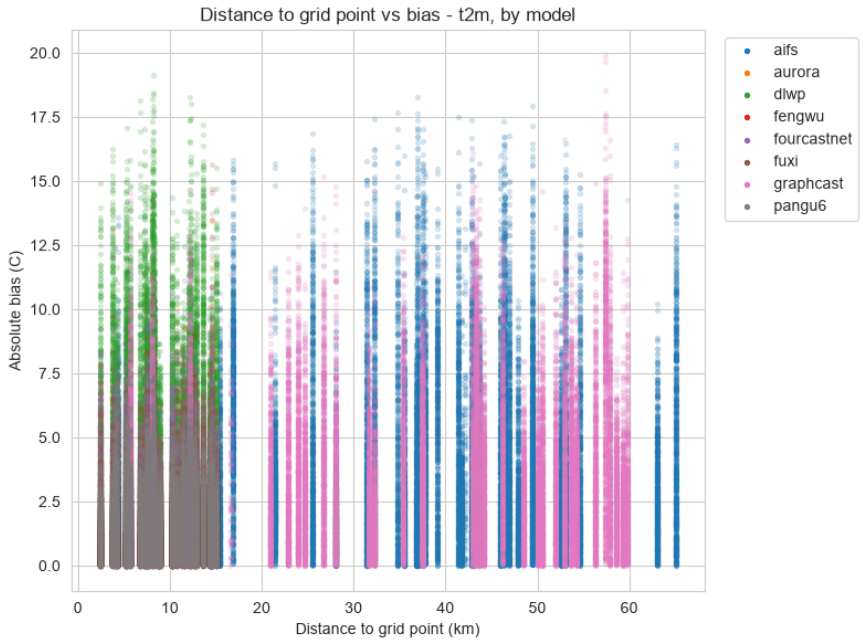


Figure 4.9: *Absolute temperature bias against the distance from the station to the grid point, coloured by AI model. The two groups of points come from models with different grid geometries, which creates a misleading overall correlation.*

4.4 Features that explain the bias

The mutual information scores effectively categorize the eight models into two distinct families (Figure 4.10), providing the foundational mathematical logic for the subsequent calibration methodology:

- **Time-driven models** (AIFS, DLWP, GraphCast): lead time and hour carry the most information, with scores between 0.25 and 0.37 against 0.14 to 0.16 for the station.
- **Location-driven models** (Aurora, FengWu, FourCastNet, FuXi, Pangu6): station, latitude and longitude dominate, with scores between 0.12 and 0.22, while hour and lead time stay below 0.03.

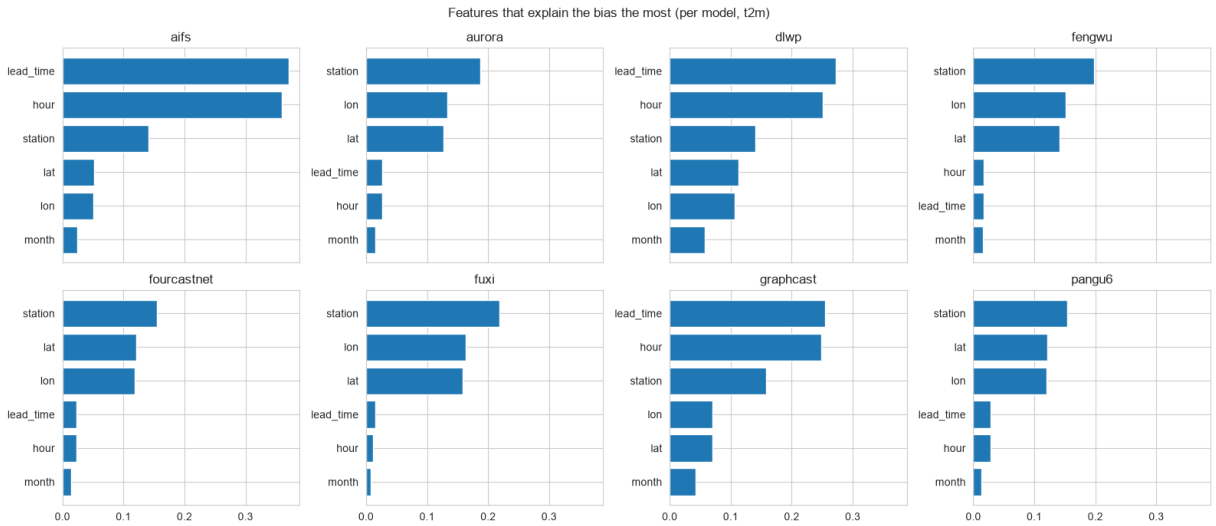


Figure 4.10: *Mutual information between each candidate feature and the temperature bias, one panel per AI model.*

The same three models appear again on the time side, which is coherent with the ANOVA results and with the hourly plots. This is what led to the four time features described in Chapter 3.

4.5 Calibration results for temperature

Config A

Table 4.2 gives, for each AI model, the best of the four algorithms in Config A. The full table with the four algorithms is in Appendix A.1.

Table 4.2: *Config A (single variable), temperature. Best algorithm per AI model, on the most recent 20% of the data. The raw RMSE is computed on these test rows only, which is why it differs from Table 4.1, computed over the whole period.*

Model	Best algorithm	R^2	Raw RMSE	Corrected RMSE	Gain
AIFS	MLP	0.694	5.30	2.93	44.7%
DLWP	Random Forest	0.542	4.39	2.88	34.5%
GraphCast	MLP	0.337	4.37	3.40	22.3%
FuXi	Random Forest	0.335	2.25	1.80	19.9%
FengWu	LightGBM	0.344	2.11	1.71	19.0%
Aurora	LightGBM	0.317	2.01	1.64	18.3%
FourCastNet	LightGBM	0.258	2.28	1.94	15.1%
Pangu6	LightGBM	0.259	2.11	1.80	14.7%

The calibration process improved all evaluated models, reducing RMSE by approximately 18% even for the highly accurate Aurora and FengWu.

Algorithm superiority strictly aligns with the model families identified in Section 4.4. The MLP excels on time driven models by capturing nonlinear temporal relationships, whereas Random Forest and LightGBM dominate on location driven models. SVR consistently underperformed across all experiments. Its optimized ε margin of 0.483 mathematically ignores errors under half a degree, removing any objective incentive to correct minor biases.

Predictive performance is notably higher for AIFS (R^2 of 0.694) and DLWP (0.542) compared to the remaining models (0.258 to 0.344). Because massive systematic errors provide a stronger learning signal than random statistical noise, models with the largest initial flaws inherently achieve the greatest relative improvements. Consequently, while AIFS secured the highest percentage gain, its absolute post calibration RMSE of 2.93°C remains among the highest due to its severe raw starting error.

Config B

Adding the other predicted variables helps, but not everywhere (Table 4.3). AIFS gains the most, moving from 44.7% to 49.0%, and its R^2 reaches 0.739, the best result of the whole project. GraphCast also improves, from 22.3% to 24.6%. The five other models stay close to their Config A values, and the MLP even loses ground: its gain drops to about 5% on Aurora, FengWu and Pangu6, while Random Forest and LightGBM keep their level.

Table 4.3: *Config B (multi-parameter), temperature. Best algorithm per AI model. DLWP is missing because it provides temperature only. The raw RMSE is again computed on the test rows, here restricted to the runs where every variable of the model is available, so it differs from both other tables.*

Model	Best algorithm	R^2	Raw RMSE	Corr. RMSE	Gain	Train rows
AIFS	Random Forest	0.739	4.93	2.51	49.0%	842
GraphCast	LightGBM	0.341	4.21	3.17	24.6%	5,543
FuXi	Random Forest	0.315	2.21	1.81	18.3%	5,705
FengWu	Random Forest	0.330	2.05	1.68	18.1%	5,549
Aurora	LightGBM	0.292	1.96	1.63	16.7%	5,712
FourCastNet	LightGBM	0.250	2.33	1.99	14.4%	2,291
Pangu6	LightGBM	0.255	2.14	1.83	14.5%	5,549

The AIFS multiparameter results require careful interpretation due to sample size constraints. Its Config B table has 842 training rows and only 211 test rows, because a row survives only if all five variables are available for the same station, run and lead time. The 49.0% is measured on a small and recent sample, so it should be read as promising rather than as an established number.

4.6 Precipitation

The overwhelming prevalence of zero values in the precipitation data necessitates a distinct modeling methodology and restricts the scope of the resulting statistical claims.

The starting point is the statistical imbalance shown in Figure 4.11: 98.48% of the tp06 observations contain no rain, and only 1.3% to 2.0% of the rows have a bias above 0.1 mm.

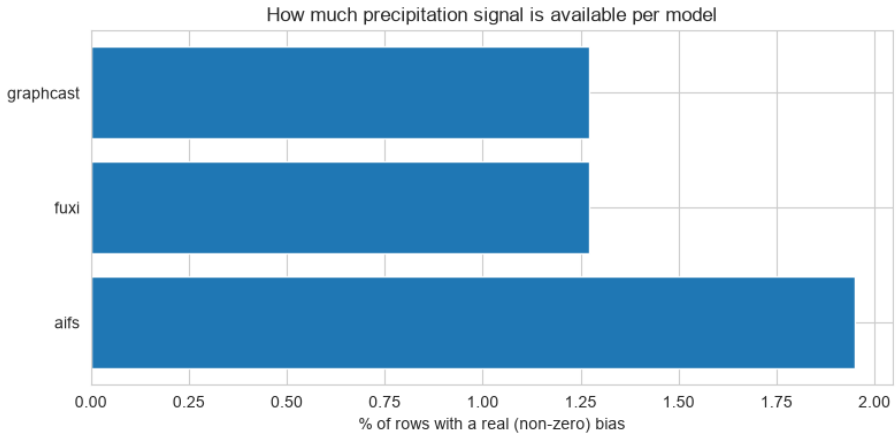


Figure 4.11: *Percentage of rows with a precipitation bias above 0.1 mm, for the three models that forecast tp06.*

Step 1: detecting the significant cases

The classifier finds most of the significant cases: recall goes from 0.50 to 0.88 depending on the model and the algorithm. Precision never rises above 0.032, which means that for one real case detected there are at least thirty false alarms. With such a rare event this was predictable, and it is the main limit of the approach.

Step 2: predicting the value of the bias

On the regression step, Random Forest reaches an R^2 of 0.17 to 0.22 in Config A. LightGBM reaches 0.22 on AIFS but stays below 0.10 on FuXi and GraphCast. Nine of the twenty experiments have a negative R^2, so the model performs worse than predicting the mean bias. Config B is clearly worse than Config A here, because filtering on all variables leaves 81 or 82 training rows and 8 test rows, which is too few to conclude anything.

The detailed performance metrics for precipitation are provided in Table A.3 of the Appendix. It is important to note that these metrics are computed strictly on the small subset of significant cases, making them fundamentally incomparable with the temperature results. Furthermore, AIFS Config B was skipped completely: only 15 significant cases were left in the training set, below the threshold of 20 required for training.

4.7 Consistency between the learned models and the exploratory analysis

Following model training, I cross referenced the feature importance rankings derived from the tree based models with the mutual information scores computed during the exploratory phase.

Random Forest agrees entirely with the exploratory analysis. For AIFS, `is_afternoon` and `hour_cos` rank highest. For DLWP and GraphCast, `is_night` leads. For the five location driven models, latitude and longitude clearly dominate (Figure 4.12).

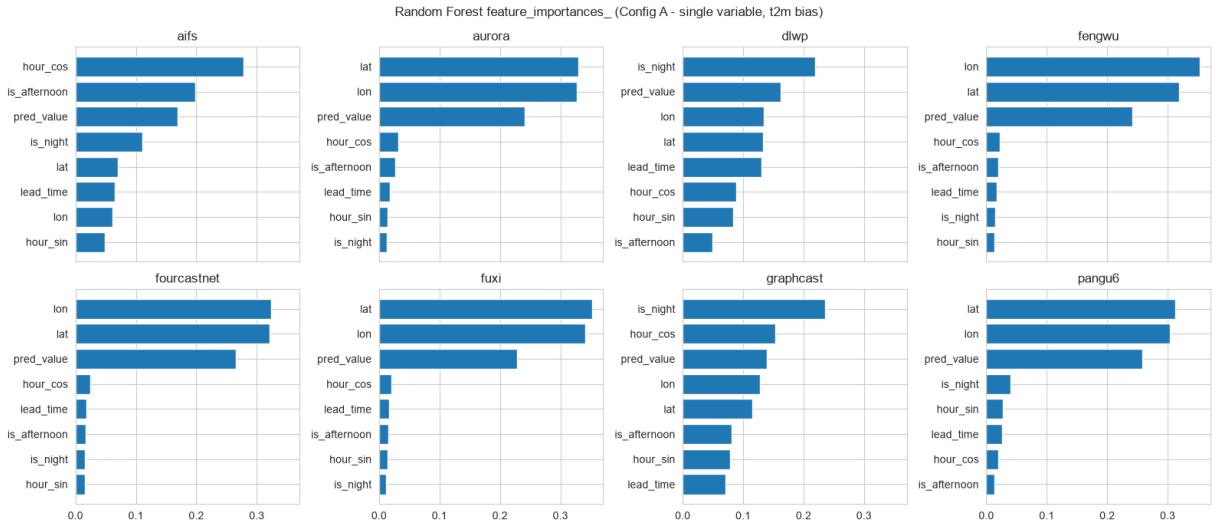


Figure 4.12: *Random Forest feature importance in Config A, one panel per AI model.*

LightGBM yields a divergent feature importance profile (Figure 4.13). It consistently ranks the predicted value, latitude, and longitude first, while pushing the temporal features downward. For AIFS, the model whose bias follows the hour more than any other, `is_night` and `is_afternoon` sit at the bottom of the panel with almost no split count at all. The likeliest explanation is technical: boosting favours continuous features with many possible split points, while a binary feature such as `is_night` offers a single split, and the importance reported by LightGBM counts how often a feature was used rather than how much the prediction depends on it. Random Forest, which selects features at random in each tree, gives the binary features more chances to be used.

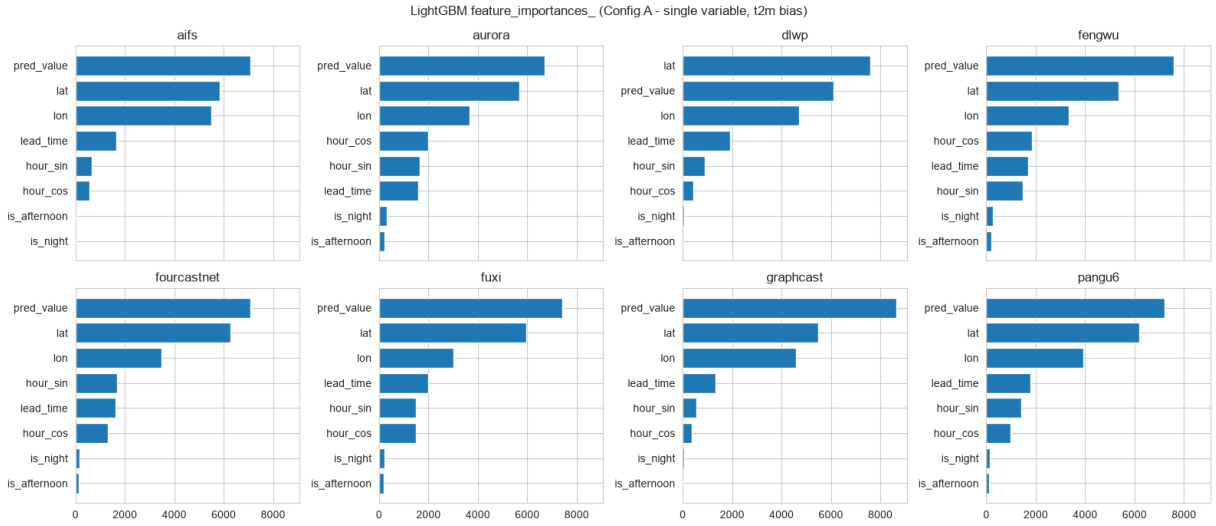


Figure 4.13: *LightGBM feature importance in Config A, one panel per AI model, on the same experiments as Figure 4.12. The two methods disagree about the binary time features.*

This discrepancy does not indicate that one method is fundamentally incorrect. Because two independent mathematical methods point in the exact same direction, I retained their shared conclusion while noting that specific boosting importance metrics must be

interpreted with caution in this context. The MLP and the SVR were left out of this comparison, because they do not provide a direct importance score.

4.8 Summary of the results

Main outcome. Eight AI weather models were evaluated at 34 Moroccan stations. Their temperature error splits them into two groups, and their bias is driven either by the hour of the day (AIFS, DLWP, GraphCast) or by the station location (the five others). Machine learning removed 14.7% to 44.7% of the temperature RMSE in the single-variable configuration, and up to 49.0% for AIFS when the other predicted variables were added. Precipitation remains uncorrected in practice.

4.9 Comparison with published results

Contextualizing these findings against existing academic literature is essential for validating the robustness of the proposed corrections.

The mechanism behind the gain agrees with Rasp and Lerch [1]. They obtain their best score by adding station information and by allowing a nonlinear combination of the inputs. My Config A does the same thing with cruder tools, latitude and longitude instead of a learned embedding, and the ranking of the algorithms matches: the nonlinear methods win where the relation is a curve.

The magnitude of the improvement requires careful contextualization. Their 33% improvement represents a CRPS gain on an already well calibrated ensemble, whereas my 44.7% improvement represents an RMSE reduction on a raw forecast with an initial error of 5.30°C. Because these two figures address entirely different statistical questions against different baseline performances, the meaningful agreement between the two studies lies in the shared underlying mechanism rather than the exact percentage gain.

Furthermore, one specific finding from my analysis was not documented in the sources I reviewed. This observation was made possible by evaluating eight distinct models simultaneously rather than focusing on a single architecture. The models split into two families with different error structures, one driven by the hour and one driven by the station, and the best algorithm is not the same for the two families. A study built around a single AI model cannot see this split.

Finally, a methodological divergence must be noted. Rasp and Lerch produce a probabilistic correction scored by the CRPS, while my corrections produce a single number. Their approach gives the forecaster an interval, which is what an operational warning system needs. Mine does not, and the Perspectives chapter lists this as the first thing to change.

4.10 Limitations

The preceding results are strictly bound by the scope of the available dataset. Explicitly defining these limitations is critical for interpreting the operational viability of the models.

The period covers seven months, from September 2025 to March 2026. It contains one autumn and one winter, and no summer. Since the bias depends on the temperature range, the corrections learned here may behave differently in July. Figure 4.8 makes this concrete inside the period itself: two of the eight models move by more than 2°C of mean bias between two months of the record, so there is no reason to assume they would stay still across a season the record does not contain.

The test set is the most recent 20% of the period, so it is mostly late winter and early spring. This is the correct way to test a forecasting method, but it means each model is judged on one season.

The station network is unbalanced. Seventeen of the 34 stations are coastal and only four sit at altitude or beyond the Atlas (Table 3.5), while those four are the hardest ones. The correction has therefore seen many examples of the easy regime and few of the difficult one.

Config B rests on small samples for AIFS and FourCastNet. The AIFS figure of 49.0% comes from 211 test rows.

The precipitation results are weak and do not form a working method.

Finally, a calibration model was trained per AI model and per variable, using station coordinates as input. I did not test whether a model trained on some stations can correct a station it has never seen, which is what a national service would need for a location without a station.

Conclusion

The fundamental objective of this internship was to determine whether emerging artificial intelligence weather models are operationally viable over Morocco and if their inherent errors could be systematically reduced.

While highly functional, baseline performance varies significantly among the eight evaluated models. Five models forecast the 2 m temperature with an RMSE between 2.1°C and 2.35°C, whereas AIFS, DLWP, and GraphCast exhibit error margins nearly twice as large. However, all architectures share an identical structural weakness at the extremes: they consistently predict cold days to be too warm and hot days to be too cold.

Because these systematic errors follow predictable physical patterns rather than random noise, they can be successfully mitigated. For AIFS, DLWP, and GraphCast, the bias is heavily dictated by the time of day, swinging from a nocturnal overestimation to a midday underestimation of several degrees. Conversely, the error magnitude for the remaining five models depends primarily on geographic location. Both statistical testing and the autonomous learning patterns of the Random Forest algorithms confirmed this fundamental separation.

Four machine learning algorithms were trained to predict and correct these specific biases. On the test data, the temperature RMSE dropped by 14.7% to 44.7% using the single variable configuration. Incorporating additional predicted variables raised the AIFS improvement peak to 49.0%. Naturally, models exhibiting the largest initial errors achieved the most substantial percentage gains. Correcting precipitation forecasts, however, proved highly challenging. The overwhelming prevalence of zero values led the two stage method to generate excessive false alarms despite successfully detecting genuine rain events.

Beyond quantitative metrics, this project established that algorithm selection must strictly align with the underlying structure of the error. Neural networks excel when bias is a nonlinear time function, whereas tree based methods perform best for location dependent errors. Furthermore, this experience reinforced the critical importance of exploratory data analysis, demonstrating that robust visual foundations and direct operational comparisons are ultimately more valuable than isolated statistical scores like R^2.

Perspectives

Several directions follow directly from the limitations listed at the end of Chapter 4.

Collect a longer period

The most immediate enhancement requires no methodological changes: expanding the dataset. A full year, and ideally two, would cover the summer and allow a seasonal check of the corrections. It would also make Config B usable, since the small sample problem comes from the intersection of variables over a short period.

Test the correction at stations it has never seen

The current models use latitude and longitude as inputs, so they can in principle produce a correction anywhere. Whether that correction is valid at a station absent from the training set is an open question. A leave-one-station-out test would answer it, and the answer matters for any operational use.

Learn the station instead of describing it

Rasp and Lerch obtained their best results with station embeddings, a small vector learned for each station [1]. This would replace latitude and longitude by something richer, and would let a single model cover all stations instead of one model per AI model. It is the natural next step for the neural network part.

Move from a single value to a probability

All the corrections here produce one number. Operational forecasting increasingly needs an interval or a distribution. Predicting a mean and a spread, and scoring them with the CRPS as in the reference paper, would fit the way a national service issues warnings.

Rework the precipitation problem

Three changes are worth testing: adjusting the classification threshold to trade recall against precision, adding humidity and dew point as inputs where they exist, and treating the rainfall amount on a transformed scale. A longer period would also increase the number of rain events, which is the real bottleneck.

Towards an operational chain

Provided these results remain consistent over extended temporal periods, the calibration process can be fully automated. Figure P.1 outlines a proposed architecture for this operational chain, presented here strictly as a conceptual framework for future deployment rather than an implemented result.

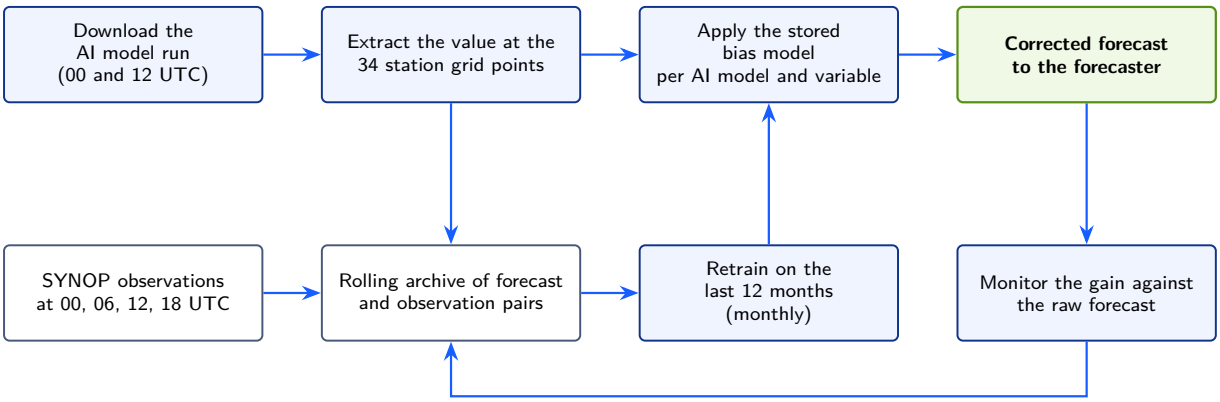


Figure P.1: *Proposed operational chain for the correction. The upper row runs with every model run; the lower row runs on a slower cycle and keeps the correction up to date.*

The primary operational risks within this proposed architecture are concentrated in two specific modules. Retraining is one: an AI weather model can be updated by its authors without warning, and a correction fitted on the previous version may then make the forecast worse instead of better. Monitoring is the other. A drop in the measured gain is the signal that something changed upstream, and it is the only cheap way to catch such a change, which is why the monitoring box feeds back into the archive rather than sitting at the end of the chain.

Bibliography

[1] Rasp, S. and Lerch, S. (2018). *Neural networks for post-processing ensemble weather forecasts*. Monthly Weather Review. Document provided with the internship materials, as a simplified summary.

[2] Guo, C., Pleiss, G., Sun, Y. and Weinberger, K. Q. (2017). *On calibration of modern neural networks*. ICML. Document provided with the internship materials, as a simplified summary.

[3] Niculescu-Mizil, A. and Caruana, R. (2005). *Predicting good probabilities with supervised learning*. ICML. Document provided with the internship materials, as a simplified summary.

[4] Glahn, H. R. and Lowry, D. A. (1972). *The use of Model Output Statistics (MOS) in objective weather forecasting*. Journal of Applied Meteorology, 11, 1203 to 1211. External source.

[5] ECMWF (2018). *Addressing biases in near-surface forecasts*. ECMWF Newsletter 157. External source.

[6] Hersbach, H. et al. (2020). *The ERA5 global reanalysis*. Quarterly Journal of the Royal Meteorological Society, 146, 1999 to 2049. External source.

[7] Ke, G. et al. (2017). *LightGBM: a highly efficient gradient boosting decision tree*. Advances in Neural Information Processing Systems 30. External source.

[8] Breiman, L. (2001). *Random forests*. Machine Learning, 45, 5 to 32. External source.

[9] Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019). *Optuna: a next-generation hyperparameter optimization framework*. KDD. External source.

[10] ECMWF (2025). *ECMWF's AI forecasts become operational*. News release, 25 February 2025. External source.

[11] Lang, S. et al. (2024). *AIFS: ECMWF's data-driven forecasting system*. External source.

[12] Lam, R. et al. (2023). *Learning skillful medium-range global weather forecasting* (GraphCast). Science. External source.

[13] Bi, K. et al. (2023). *Accurate medium-range global weather forecasting with 3D neural*

networks (Pangu-Weather). Nature. External source.

[14] Pathak, J. et al. (2022). *FourCastNet: a global data-driven high-resolution weather model using adaptive Fourier neural operators*. External source.

[15] Chen, K. et al. (2023). *FengWu: pushing the skillful global medium-range weather forecast beyond 10 days lead*. External source.

[16] Chen, L. et al. (2023). *FuXi: a cascade machine learning forecasting system for 15-day global weather forecast*. External source.

[17] Bodnar, C. et al. (2025). *Aurora: a foundation model of the atmosphere*. External source.

[18] General Directorate of Meteorology (DGM) (Maroc Météo). *Institutional presentation and organisation chart*. Internal documents provided by the host organisation.

Appendix A

Supplementary tables

Outline

A.1 Full calibration results for temperature 57
A.2 Precipitation results 59
A.3 Selected code 60
A.4 Selected hyperparameters 63

A.1 Full calibration results for temperature

Table A.1 gives every algorithm in Config A, and Table A.2 every algorithm in Config B. The gain column is the reduction in RMSE of the corrected forecast compared with the raw forecast, on the same test rows.

Table A.1: *Config A (single variable), temperature: all four algorithms for the eight AI models.*

Model	Algorithm	MAE	RMSE	R^2	Gain
AIFS	MLP	2.244	2.931	0.694	44.7%
AIFS	SVR	2.307	3.008	0.678	43.3%
AIFS	Random Forest	2.248	2.943	0.692	44.5%
AIFS	LightGBM	2.267	2.993	0.681	43.6%
Aurora	MLP	1.254	1.666	0.299	17.2%
Aurora	SVR	1.450	1.887	0.100	6.2%
Aurora	Random Forest	1.244	1.657	0.306	17.7%
Aurora	LightGBM	1.229	1.644	0.317	18.3%
DLWP	MLP	2.189	2.888	0.538	34.3%
DLWP	SVR	2.282	2.993	0.504	31.9%

Model	Algorithm	MAE	RMSE	R^2	Gain
DLWP	Random Forest	2.182	2.875	0.542	34.5%
DLWP	LightGBM	2.213	2.910	0.531	33.7%
FengWu	MLP	1.315	1.749	0.311	17.0%
FengWu	SVR	1.514	1.973	0.122	6.4%
FengWu	Random Forest	1.285	1.707	0.343	19.0%
FengWu	LightGBM	1.278	1.706	0.344	19.0%
FourCastNet	MLP	1.516	1.972	0.230	13.5%
FourCastNet	SVR	1.663	2.151	0.084	5.7%
FourCastNet	Random Forest	1.495	1.948	0.248	14.6%
FourCastNet	LightGBM	1.484	1.936	0.258	15.1%
FuXi	MLP	1.384	1.819	0.323	19.1%
FuXi	SVR	1.591	2.063	0.129	8.2%
FuXi	Random Forest	1.364	1.802	0.335	19.9%
FuXi	LightGBM	1.365	1.806	0.332	19.7%
GraphCast	MLP	2.634	3.396	0.337	22.3%
GraphCast	SVR	2.761	3.532	0.283	19.2%
GraphCast	Random Forest	2.655	3.427	0.325	21.6%
GraphCast	LightGBM	2.703	3.455	0.314	20.9%
Pangu6	MLP	1.430	1.869	0.199	11.3%
Pangu6	SVR	1.553	1.996	0.086	5.2%
Pangu6	Random Forest	1.375	1.804	0.254	14.4%
Pangu6	LightGBM	1.368	1.797	0.259	14.7%

Table A.2: *Config B (multi-parameter), temperature: all four algorithms for the seven AI models that publish more than one variable.*

Model	Algorithm	MAE	RMSE	R^2	Gain
AIFS	MLP	2.036	2.540	0.733	48.4%
AIFS	SVR	2.070	2.658	0.707	46.0%
AIFS	Random Forest	1.967	2.512	0.739	49.0%
AIFS	LightGBM	1.982	2.608	0.718	47.1%
Aurora	MLP	1.449	1.866	0.071	4.6%
Aurora	SVR	1.408	1.831	0.105	6.4%
Aurora	Random Forest	1.244	1.646	0.277	15.9%

Model	Algorithm	MAE	RMSE	R^2	Gain
Aurora	LightGBM	1.229	1.629	0.292	16.7%
FengWu	MLP	1.520	1.957	0.091	4.7%
FengWu	SVR	1.487	1.934	0.113	5.8%
FengWu	Random Forest	1.286	1.681	0.330	18.1%
FengWu	LightGBM	1.291	1.691	0.322	17.6%
FourCastNet	MLP	1.639	2.117	0.154	9.0%
FourCastNet	SVR	1.683	2.185	0.099	6.1%
FourCastNet	Random Forest	1.567	2.000	0.245	14.1%
FourCastNet	LightGBM	1.552	1.993	0.250	14.4%
FuXi	MLP	1.468	1.899	0.243	14.1%
FuXi	SVR	1.599	2.060	0.109	6.8%
FuXi	Random Forest	1.371	1.806	0.315	18.3%
FuXi	LightGBM	1.377	1.813	0.310	18.0%
GraphCast	MLP	2.645	3.394	0.246	19.3%
GraphCast	SVR	2.569	3.282	0.295	22.0%
GraphCast	Random Forest	2.522	3.193	0.332	24.1%
GraphCast	LightGBM	2.495	3.173	0.341	24.6%
Pangu6	MLP	1.589	2.031	0.081	5.0%
Pangu6	SVR	1.555	2.003	0.106	6.3%
Pangu6	Random Forest	1.406	1.836	0.249	14.1%
Pangu6	LightGBM	1.407	1.829	0.255	14.5%

A.2 Precipitation results

Table A.3 gives the two-step results for the three models that forecast precipitation. The classification scores come from step 1 and the regression scores from step 2, computed only on the rows with a significant bias. AIFS in Config B is absent: only 15 significant cases remained in the training set, below the minimum of 20 required to train.

Table A.3: *Precipitation (tp06) results. Precision, recall and F1 come from the classification step; MAE, RMSE and R^2 from the regression step.*

Model	Config	Algorithm	Prec.	Recall	F1	MAE	RMSE	R^2
AIFS	A	MLP	0.007	0.850	0.014	1.538	2.368	0.050
AIFS	A	SVR	0.009	0.750	0.018	1.549	2.438	−0.008
AIFS	A	Random Forest	0.013	0.850	0.027	1.373	2.208	0.174
AIFS	A	LightGBM	0.015	0.850	0.030	1.280	2.151	0.216
FuXi	A	MLP	0.021	0.600	0.040	1.780	2.438	−0.008
FuXi	A	SVR	0.023	0.700	0.045	1.710	2.489	−0.050
FuXi	A	Random Forest	0.022	0.750	0.044	1.496	2.143	0.222
FuXi	A	LightGBM	0.023	0.750	0.044	1.612	2.317	0.090
FuXi	B	MLP	0.009	0.875	0.018	1.893	3.170	0.187
FuXi	B	SVR	0.022	0.625	0.042	1.933	3.568	−0.030
FuXi	B	Random Forest	0.016	0.625	0.031	1.973	3.422	0.053
FuXi	B	LightGBM	0.023	0.500	0.044	2.045	3.557	−0.023
GraphCast	A	MLP	0.008	0.650	0.016	1.721	2.394	0.028
GraphCast	A	SVR	0.022	0.650	0.043	1.702	2.475	−0.038
GraphCast	A	Random Forest	0.023	0.750	0.044	1.471	2.186	0.190
GraphCast	A	LightGBM	0.020	0.600	0.040	1.624	2.347	0.066
GraphCast	B	MLP	0.007	0.875	0.013	2.138	3.279	0.130
GraphCast	B	SVR	0.014	0.625	0.028	2.008	3.592	−0.044
GraphCast	B	Random Forest	0.019	0.875	0.037	2.175	3.553	−0.021
GraphCast	B	LightGBM	0.032	0.750	0.062	2.053	3.554	−0.022

A.3 Selected code

The listings below are taken from the analysis notebook. They cover the time-ordered split, the comparison with the raw forecast and the first step of the precipitation method, which are the three pieces the results of Chapter 4 depend on. Imports and plotting calls are omitted.

Splitting on time

Every result in Chapter 4 rests on this function. The table is sorted on the valid time and cut at 80%, so no row from the test period can reach the training set. No random shuffling is involved at any point.

Listing A.1: *Feature list and the time-ordered train and test split.*

```
BASE_FEATURES = ["lat", "lon", "lead_time",
```

```
                  "hour_sin", "hour_cos", "is_afternoon", "is_night"]
FEATURES = ["pred_value"] + BASE_FEATURES

def split_train_test(table, test_fraction=0.2):
    table = table.sort_values("valid_time")
    n_rows = len(table)
    n_train = int(n_rows * (1 - test_fraction))
    train_part = table.iloc[:n_train].copy()
    test_part = table.iloc[n_train:].copy()
    return train_part, test_part
```

Measuring the gain against the raw forecast

Every improvement percentage in the report comes from this function, including the 44.7% and the 49.0%. No model is ever compared with another model: the raw forecast and the corrected forecast are compared with the same observations, on the same test rows.

Listing A.2: *Comparison of the corrected forecast with the raw forecast on the test rows.*

```
def compare_to_baseline(test_table, predicted_bias, pred_column="pred_value"):
    real_obs = test_table["obs_value"]

    raw_forecast = test_table[pred_column]
    corrected_forecast = test_table[pred_column] - predicted_bias

    raw_mae = mean_absolute_error(real_obs, raw_forecast)
    raw_rmse = np.sqrt(mean_squared_error(real_obs, raw_forecast))

    corrected_mae = mean_absolute_error(real_obs, corrected_forecast)
    corrected_rmse = np.sqrt(mean_squared_error(real_obs, corrected_forecast))

    rmse_improvement_pct = (1 - corrected_rmse / raw_rmse) * 100
    mae_improvement_pct = (1 - corrected_mae / raw_mae) * 100

    return {
        "raw_MAE": round(raw_mae, 3),
        "raw_RMSE": round(raw_rmse, 3),
        "corrected_MAE": round(corrected_mae, 3),
        "corrected_RMSE": round(corrected_rmse, 3),
        "RMSE_improvement_pct": round(rmse_improvement_pct, 1),
        "MAE_improvement_pct": round(mae_improvement_pct, 1),
    }
```

The first step for precipitation

The extract below is step 1 of the precipitation method described in Section 3.12. Two details in it explain the results of Section 4.6: the threshold of 0.1 mm that defines a significant bias, and the minimum of 20 positive cases required before training, which is why AIFS has no Config B line in the precipitation table.

Listing A.3: *Step 1 of the precipitation method (extract): labelling of the significant cases and weighting of the rare class.*

```
def two_step_precipitation(weather_model, config="single",
                           algorithm="mlp", threshold=0.1):
    if config == "single":
        subset = df[(df["model"] == weather_model) &
                    (df["variable_name"] == "tp06")]
        features = FEATURES
        train, test = split_train_test(subset)
    else:
        other_variables = get_other_variables(weather_model, "tp06")
        wide_table, features = build_multi_parameter_table(
            weather_model, "tp06", other_variables)
        train, test = split_train_test(wide_table)

    # Step 1 target: is the precipitation bias significant or not?
    train_label = (train["biais"].abs() > threshold).astype(int)
    test_label = (test["biais"].abs() > threshold).astype(int)

    n_positive = train_label.sum()
    if n_positive < 20:
        print("  Not enough examples to train. Skipping this model.")
        return

    # The rare class is weighted by the ratio between the two classes.
    n_negative = len(train_label) - n_positive
    positive_weight = n_negative / n_positive
    sample_weight = np.where(train_label == 1, positive_weight, 1.0)

    # The classifier is then fitted on train[features] with these weights.
    # Step 2 fits a regression on the rows whose label is 1, and the two
    # steps are scored separately in Table A.3.
```

A.4 Selected hyperparameters

Table A.4 gives the values found by the Optuna searches described in Chapter 3. Config B reuses the Config A values for the MLP, the SVR and LightGBM, because its own search generalised worse.

Table A.4: *Hyperparameters used for the calibration models.*

Algorithm	Values	Source
MLP	`hidden_layer_sizes=(64,32)`, `activation="tanh"`, `alpha=0.000292`, `learning_rate_init=0.000817`	Config A search
SVR	`kernel="rbf"`, `C=0.297`, `epsilon=0.483`, `gamma=0.0785`	Config A search
Random Forest (A)	`n_estimators=179`, `max_depth=22`, `min_samples_split=5`, `min_samples_leaf=10`, `max_features=0.360`, `max_samples=0.602`	Config A search
Random Forest (B)	`n_estimators=459`, `max_depth=20`, `min_samples_split=15`, `min_samples_leaf=6`, `max_features=0.583`, `max_samples=0.981`	Config B search
LightGBM	`n_estimators=250`, `max_depth=12`, `learning_rate=0.0115`, `num_leaves=88`, `subsample=0.675`, `min_child_samples=60`, `colsample_bytree=0.753`, `reg_alpha=0.0033`, `reg_lambda=0.0061`	Config A search

Step 1 of the precipitation method uses classifiers, not regressors, so it has its own set of values (Table A.5). Two differences with the table above are worth noticing. The Random Forest classifier is much shallower, with a depth of 4 against 22, and it needs at least 25 samples in a leaf: on 442 training rows carrying a significant bias, a deep tree would memorise them. The SVC runs with a large C, which pushes it to fit the rare class instead of ignoring it, and it is used together with the sample weights described in Section 3.12.

Table A.5: *Hyperparameters of the classifiers used in step 1 of the precipitation method.*

Algorithm	Values	Class imbalance
MLP classifier	`hidden_layer_sizes=(16,)`, `alpha=0.000017`, `learning_rate_init=0.00278`, `early_stopping=True`	Oversampling
SVC	`kernel="rbf"`, `C=148.97`, `gamma=0.00876`	Sample weights
Random Forest	`n_estimators=462`, `max_depth=4`, `min_samples_split=36`, `min_samples_leaf=25`, `max_features=0.778`, `max_samples=0.993`	Sample weights
LightGBM	`n_estimators=546`, `max_depth=9`, `learning_rate=0.00528`, `num_leaves=25`, `subsample=0.978`, `min_child_samples=80`, `colsample_bytree=0.740`, `reg_alpha=1.378`, `reg_lambda=0.317`	Sample weights